algorithms

MDPI

Article

Q-Learning-Based Multi-Strategy Topology Particle Swarm Optimization Algorithm

Xiaoxi Hao [1], Shenwei Wang [1], Xiaotong Liu [1], Tianlei Wang [1,*], Guangfan Qiu [1] and Zhiqiang Zeng [2]

[1] School of Mechanical and Automation Engineering, Wuyi University, Jiangmen 529020, China; hxx_j001883@wyu.edu.cn (X.H.); 2112304111@wyu.edu.cn (S.W.); 2112304011@wyu.edu.cn (X.L.); 2112304125@wyu.edu.cn (G.Q.)

[2] School of Mechanical Engineering, Dongguan University of Technology, Dongguan 523808, China; zhiqiang.zeng@dgut.edu.cn

* Correspondence: 2112424044@wyu.edu.cn

Abstract

In response to the issues of premature convergence and insufficient parameter control in Particle Swarm Optimization (PSO) for high-dimensional complex optimization problems, this paper proposes a Multi-Strategy Topological Particle Swarm Optimization algorithm (MSTPSO). The method builds upon a reinforcement learning-driven topological switching framework, where Q-learning dynamically selects among fully informed topology, small-world topology, and exemplar-set topology to achieve an adaptive balance between global exploration and local exploitation. Furthermore, the algorithm integrates differential evolution perturbations and a global optimal restart strategy based on stagnation detection, together with a dual-layer experience replay mechanism to enhance population diversity at multiple levels and strengthen the ability to escape local optima. Experimental results on 29 CEC2017 benchmark functions, compared against various PSO variants and other advanced evolutionary algorithms, show that MSTPSO achieves superior fitness performance and exhibits stronger stability on high-dimensional and complex functions. Ablation studies further validate the critical contribution of the Q-learning-based multi-topology control and stagnation detection mechanisms to performance improvement. Overall, MSTPSO demonstrates significant advantages in convergence accuracy and global search capability.

Keywords: particle swarm optimization; multi-topology structure; reinforcement learning; experience replay strategy; stagnation detection

check for updates

Academic Editor: Boting Yang

Received: 22 August 2025
Revised: 1 October 2025
Accepted: 13 October 2025
Published: 22 October 2025

Citation: Hao, X.; Wang, S.; Liu, X.; Wang, T.; Qiu, G.; Zeng, Z. Q-Learning-Based Multi-Strategy Topology Particle Swarm Optimization Algorithm. *Algorithms* **2025**, *18*, 672. https://doi.org/10.3390/a18110672

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Optimization techniques play a central role in fields such as engineering design, machine learning, artificial intelligence, and operations research. From parameter optimization of complex mechanical structures to hyperparameter tuning of deep learning models, efficient optimization algorithms are the key to solving practical problems. Current mainstream optimization algorithms include differential evolution (DE) [1,2], Genetic Algorithm (GA) [3], Bayesian Optimization (BO) [4], and Artificial Bee Colony (ABC) algorithm [5], among which Particle Swarm Optimization (PSO) has been widely applied due to its simple structure, easy parameter adjustment, and convenient implementation [6–8]. Since its introduction by Kennedy and Eberhart in 1995, PSO has simulated the social behavior of biological populations to perform optimization search and has spawned numerous improved variants [6]. However, traditional PSO still suffers from significant limitations: most variants are effective only for specific problem types, PSO tends to fall into local

optima in high-dimensional multimodal functions, and fixed topology structures limit population diversity [9,10]. To overcome these bottlenecks, researchers have systematically improved PSO in several directions.

First, hybrid strategies integrate the strengths of different algorithms to enhance search performance. Recent studies include GWOPSO, which combines Grey Wolf Optimization (GWO) with PSO by leveraging PSO's population collaboration mechanism and GWO's hierarchical hunting strategy [11]; ECCSPSOA, which integrates chaotic crow search with PSO to improve the diversity of initial solutions for feature selection problems [12]; and HGSPSO, which merges gravitational search with PSO to optimize particle interactions via a gravity model [13]. In particular, hybridization of PSO and differential evolution (DE) has become a research hotspot: heterogeneous DE-PSO improves exploitation through local search [14], while Mutual Learning DE-PSO enhances global exploration by knowledge sharing among swarms [15]. These hybrid approaches have demonstrated significant advantages in complex function optimization tasks, as validated on CEC2017 benchmarks [16,17].

Second, learning strategies focus on building efficient guidance mechanisms. Liu et al. proposed simplified PSO models with cognitive, social, and temporal-hierarchical strategies (THSPSO), which dynamically update learning patterns to improve optimization efficiency and significantly shorten convergence time compared with standard PSO in complex function optimization [10]. Huang et al. introduced the LRMODE algorithm, which incorporates ruggedness analysis of the fitness landscape to refine particle learning directions [18]. Lu et al. employed reinforcement learning to guide particle behavior, integrating historical successful experiences into wastewater treatment control optimization [19]. Zhao et al. proposed the mean-hierarchy PSO (MHPSO), which stratifies particles according to population mean fitness and uses stochastic elite neighbor selection to maintain diversity [20]. Wang et al. developed the SCDLPSO algorithm with self-correction and dimension-wise learning ability to strengthen links between individual and global bests for solving complex multimodal problems [21]. Other studies have enhanced adaptability at different stages by introducing adaptive inertia weights and learning factors [22,23]. These methods dynamically adjust learning exemplars to strengthen PSO's adaptability to complex problems.

Third, neighborhood topology innovations are critical to improving PSO performance. While traditional global topology risks premature convergence and local topology converges slowly [24], dynamic topologies provide balance. For example, SWD-PSO employs small-world networks with dense short-range links and random long-range connections to accelerate global information exchange [25]. DNSPSO adapts neighborhood range via dynamic switching mechanisms [26], while MNPSO applies distance-adaptive neighborhoods to improve nonlinear equation solving [27]. Pyramid PSO (PPSO) enhances population diversity through competition-cooperation strategies and multi-behavior learning [28]. Liang et al. proposed NRLPSO, a neighborhood differential mutation PSO guided by reinforcement learning, with a dynamic oscillatory inertia weight to adapt particle motion [29]. Jiang et al. developed HPSO-TS, a hybrid PSO with time-varying topology and search perturbation, which uses K-medoids clustering to dynamically divide the swarm into heterogeneous subgroups, facilitating intra-group information flow and global-local search transitions [30]. Zeng et al. further proposed a dynamic-neighborhood switching PSO (DNSPSO) with a novel velocity update rule and switching strategy, showing excellent performance on multimodal optimization problems [26]. These topology improvements have greatly enhanced search capability in high-dimensional spaces.

Finally, adaptive parameter control in PSO has evolved from heuristic rules to intelligent learning strategies. Tatsis et al. introduced an online adaptive parameter method that

combines performance estimation with gradient search, significantly improving robustness in complex environments [31]. Chen et al. developed MACPSO, which applies multiple Actor-Critic networks to optimize parameters in continuous space, balancing global exploration and local exploitation [16]. Liu et al. proposed QLPSO, where Q-learning trains parameters and Q-tables guide particle actions based on states, improving search efficiency in complex solution spaces [32]. A deep reinforcement learning-based PSO (DRLPSO) [33] employs neural networks to learn state-action mappings and dynamically adjust parameters for high-dimensional tasks. Hamad et al. proposed QLESCA, which introduces Q-learning into the sine cosine algorithm (SCA) to achieve adaptive tuning of key parameters, significantly improving convergence speed and solution accuracy. Subsequent studies applied QLESCA to high-dimensional COVID-19 feature selection, verifying its potential in high-dimensional problems; however, the overall optimization strategy remains primarily parameter-focused [34,35]. RLPSO algorithm employs a reinforcement learning strategy to adaptively control the hierarchical structure of the swarm, adjusting search modes across different levels to balance exploration and exploitation [36]. Most existing RL-PSO methods still concentrate on parameter-level adaptation. Although this improves convergence performance to some extent, little work has addressed adaptive control of the swarm topology. As topology determines the interaction pattern among particles, it plays a crucial role in maintaining population diversity and balancing exploration and exploitation. Therefore, introducing reinforcement learning for dynamic topology selection remains an area requiring further investigation. Based on this background, the main contributions of this paper are as follows:

- A reinforcement learning-based topology switching strategy is proposed, enabling particles to dynamically select among FIPS, small-world, and exemplar-set topologies to balance global exploration and local exploitation.
- A dual-layer Q-learning experience replay mechanism is designed, integrating short-term and long-term memories to stabilize parameter control and improve learning efficiency.
- A stagnation detection mechanism is constructed and combined with differential evolution perturbations and a global restart strategy to enhance population diversity and improve the ability to escape local optima.

Compared with existing RL-PSO methods that mainly focus on parameter adaptation, the proposed MSTPSO applies reinforcement learning to adaptive control of swarm topology, optimizing the information exchange pattern at the structural level and thereby improving convergence efficiency and robustness. The remainder of this paper is organized as follows: Section 2 reviews PSO and reinforcement learning–related theories. Section 3 describes the design and implementation of MSTPSO. Section 4 presents performance validation on the CEC2017 benchmark and ablation analysis of each component. Section 5 concludes the study and outlines future research directions.

2. Background Information

2.1. Particle Swarm Optimization

The Particle Swarm Optimization (PSO) algorithm was proposed by Kennedy and Eberhart in 1995 [7], inspired by the foraging behavior of bird flocks. In PSO, each individual in the population is abstracted as a "particle", and each particle's position corresponds to a potential solution to the optimization problem. Through cooperation and information sharing within the swarm, global optimal search can be achieved. In a standard PSO, a particle swarm consisting of N particles is described by two vector quantities: velocity vector v and position vector x. For a D-dimensional search space, the velocity and position of the i-th particle are defined as

$$v_i = (v_{i1}, v_{i2}, \ldots, v_{iD}) \in D, \quad i = 1, 2, \ldots, N. \tag{1}$$

$$x_i = (x_{i1}, x_{i2}, \ldots, x_{iD}) \in D, \quad i = 1, 2, \ldots, N. \tag{2}$$

Equation (1) represents the velocity vector of the particle in D-dimensional space, while Equation (2) denotes its position in the search space. Each particle has a memory ability and can store the best position it has visited so far, referred to as the individual best position $pb_i(h)$. Among all individual bests $pb_i(h)$, the best one is selected as the global best position $gb(h)$. The velocity and position of particles are updated iteratively using the following equations:

$$v_i(h+1) = \omega v_i(h) + c_1 r_1 \big(pb_i(h) - x_i(h)\big) + c_2 r_2 \big(gb(h) - x_i(h)\big). \tag{3}$$

$$x_i(h+1) = x_i(h) + v_i(h+1). \tag{4}$$

In the above: ω is the inertia weight, which maintains the particle's previous momentum and balances exploration and exploitation; c_1 and c_2 are cognitive and social acceleration coefficients, representing the particle's self-learning and swarm-learning abilities, respectively; $r_1, r_2 \sim [0, 1]$ are random numbers uniformly distributed within $[0, 1]$, introducing stochasticity to avoid premature convergence; h is the iteration index; $v_i(h)$ and $x_i(h)$ represent the velocity and position of the i-th particle at the h-th iteration, respectively. The PSO mechanism is driven by a dual principle of "individual experience + group cooperation", enabling particles to update their positions by considering both their historical best positions $pb_i(h)$ and the global best position $gb(h)$, thereby forming a self-organizing search process. This mechanism allows PSO to dynamically balance global exploration and local exploitation through a simple structure and parameter configuration. However, traditional PSO often uses fixed parameter settings, which limits its adaptability. This paper designs an adaptive multi-strategy inertia weight adjustment mechanism based on particle state feedback to achieve dynamic balance. Moreover, coefficients such as c_1 and c_2 are linearly adjusted to enhance convergence capability and address the parameter sensitivity issue in standard PSO.

2.2. Reinforcement Learning

Reinforcement learning (RL) is a key branch of machine learning that focuses on enabling intelligent agents to make sequential decisions in dynamic environments. Its core objective is to learn optimal or near-optimal policies through interactions with the environment to maximize expected cumulative long-term rewards. This process can be modeled as a Markov Decision Process (MDP), formally defined as a quadruple (S, A, P, R) [37], where S represents the set of states, describing all possible situations the agent may encounter; A denotes the action set available to the agent; $P : S \times A \times S \to [0, 1]$ is the state transition probability function, describing the probability of transitioning from one state to another given an action; $R : S \times A \to R$ is the reward function, quantifying the immediate feedback received by the agent for performing an action in a given state.

Through continuous interaction with the environment, the agent receives a sequence of observations and generates an experience trajectory $\{s, a, r, s', \ldots\}$, where the cumulative discounted R_t return at time t is defined as

$$R_t = \sum_{i=t}^{T} \gamma^{i-t} r_i, \quad \gamma \in [0, 1], \tag{5}$$

where γ is the discount factor, used to balance the importance between immediate and future rewards. The core objective of RL is to find the optimal policy $\pi^*(a|s)$ that maximizes the expected cumulative return

$$\pi^*(a|s) = \arg\max_a \mathbb{E}[R_t \mid s, a]. \tag{6}$$

To achieve this, RL commonly defines a state-value function $V^*(s)$ and an action-value function $Q^*(s, a)$, as follows:

$$V^*(s) = \mathbb{E}_t[r_{t+1} + \gamma V^*(s_{t+1}) \mid s_t = s]. \tag{7}$$

$$Q^*(s, a) = \mathbb{E}_t[r_{t+1} + \gamma Q^*(s_{t+1}, a_{t+1}) \mid s_t = s, a_t = a]. \tag{8}$$

When the optimal policy π^* is achieved, the action-value function $Q^*(s, a)$ satisfies the Bellman optimality condition, which forms the theoretical foundation of the Q-learning algorithm [38].

2.3. Q-Learning

Q-learning, a representative algorithm in reinforcement learning (RL), aims to learn the optimal action-selection policy in an environment by maximizing the expected cumulative reward. RL typically models the learning process through continuous interactions between the agent and the environment, based on three key elements: state, action, and reward. The theoretical basis of Q-learning was proposed by Watkins in 1989 [39], which focuses on estimating the value (Q-value) of performing an action in a specific state without requiring a model of the environment's dynamics. Q-learning uses an iterative update rule to approximate the optimal Q-value function, with the update equation given as

$$Q(s_{t+1}, a_{t+1}, i) = (1 - \alpha)Q(s_t, a_t, i) + \alpha\left[R(s_t, a_t) + \gamma \max_a Q(s_{t+1}, a, i)\right], \tag{9}$$

where $\alpha \in [0, 1]$ is the learning rate, controlling how much newly acquired information overrides the old one; $\gamma \in [0, 1]$ is the discount factor, controlling the importance of future rewards; r_{t+1} is the immediate reward received after performing action a_t in state s_t; $\max_a Q(s_{t+1}, a, i)$ represents the maximum estimated future return from the next state s_{t+1}, assuming the best action is taken.

This update process is an off-policy method that uses the greedy policy to update Q-values while allowing exploratory behavior during the learning process through strategies such as ϵ-greedy selection. The algorithm bootstraps its Q-value estimations, progressively improving policy performance through value iteration. The core architecture of the Q-learning model is illustrated in Figure 1, and its workflow can be summarized as a closed-loop cycle of state perception → action selection → environment interaction → reward acquisition → Q-table update. This mechanism aligns well with the dynamic parameter adjustment requirements of Particle Swarm Optimization (PSO); by defining the search states of PSO (such as particle distribution and fitness variation) as RL states and defining parameter combinations as RL actions, Q-learning enables online optimization of PSO parameters through its adaptive updating capability [40].

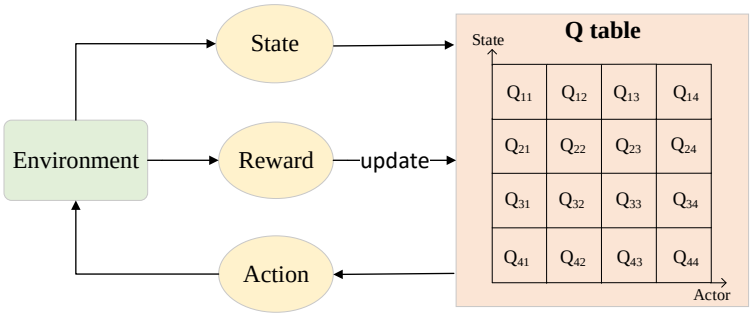


Figure 1. The model of Q-learning.

3. Multi-Strategy Topology PSO

This paper proposes a multi-topology Particle Swarm Optimization algorithm integrated with reinforcement learning, and its workflow is illustrated in Figure 2. In each iteration, the Q-learning agent selects the topology and parameter configuration according to the current state, after which particles update their velocities and positions and perform fitness evaluation. Stagnation detection is then carried out: if swarm stagnation is detected, a partial restart based on the global best is triggered, while individual stagnation activates a differential evolution–based perturbation. The Q-table is updated in every iteration using immediate feedback, whereas prioritized experience replay from the dual-layer experience pool is triggered periodically to further optimize the strategy.

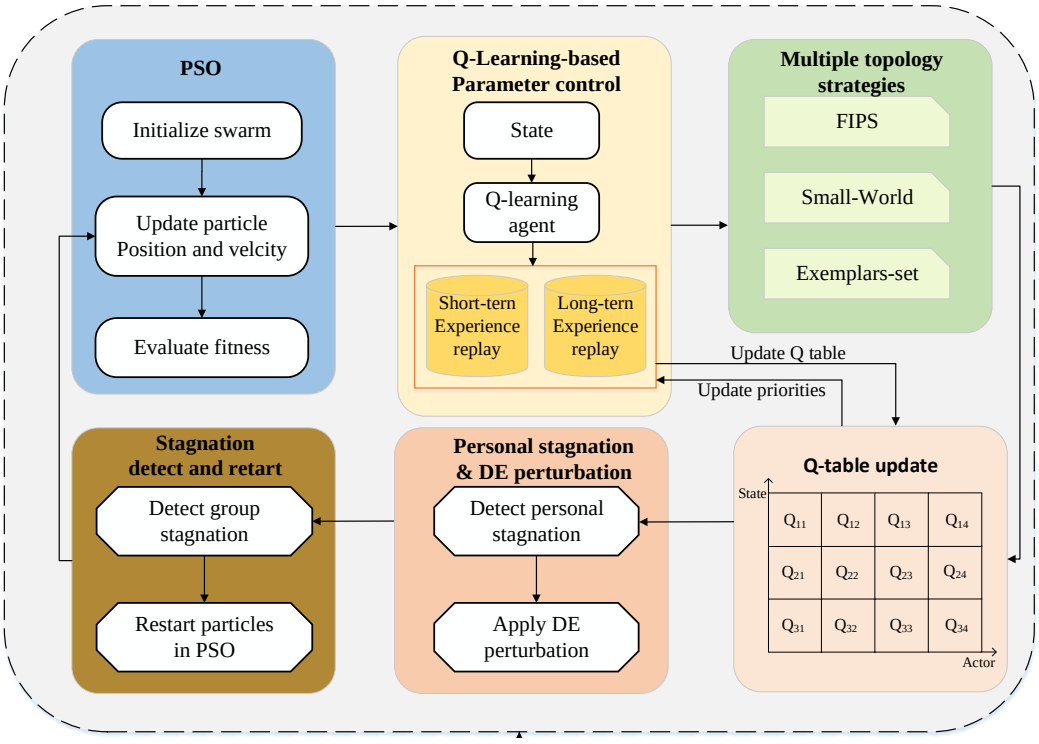


Figure 2. Algorithmic flow diagram.

3.1. State

Under the Q-learning framework, the state space needs to simultaneously reflect both the convergence trend of individual particles and the diversity of the entire swarm, thereby characterizing the dynamic balance between exploration and exploitation. In this paper, the state is designed with two complementary components: the fitness improvement state and the diversity shrinkage state. For particle i at generation t, the change in fitness is defined as

$$\Delta f_i^t = f_i^{t-1} - f_i^t, \tag{10}$$

where f_i^t denotes the fitness of particle iii at generation t. According to the value of Δf_i^t, the base state of particle i, denoted as $base_state_i$, is categorized as

$$base_state_i = \begin{cases} 1, & \Delta f_i^t < -10^{-8} \\ 2, & -10^{-8} < \Delta f_i^t < 0 \\ 3, & \Delta f_i^t = 0 \\ 4, & \Delta f_i^t > 0 \end{cases}, \tag{11}$$

where the threshold 10^{-8} is commonly used in swarm intelligence optimization to distinguish between significant and non-significant improvements, ensuring the stability and robustness of numerical judgment. The states correspond respectively to significant degradation, slight degradation, no change, and improvement. To quantify the diversity of the swarm distribution, Shannon entropy is introduced,

$$H_i^t = -\sum_{i=1}^{M} p_i \cdot \log_2(p_i), \tag{12}$$

where M = 10 denotes the number of partitions of the search space, and represents the proportion of particles falling into the j-th interval. According to the entropy value, the diversity level is categorized as

$$s_i = (entropy_state_i - 1) \times 4 + base_state_i. \tag{13}$$

These correspond to low, medium, and high diversity, respectively. Shannon entropy has been widely applied in PSO to measure the distribution and convergence behavior of the swarm [32].

3.2. Action Design

This study selects three representative topologies as reinforcement learning actions: Fully-Informed Particle Swarm Topology (FIPS) [41], small-world topology [25], and exemplar-set topology [42]. Figure 3 shows the three topological structures adopted in this work. These topologies have been demonstrated to significantly enhance global searchability and convergence efficiency in multimodal and complex optimization problems. Their corresponding velocity update mechanisms are described below.

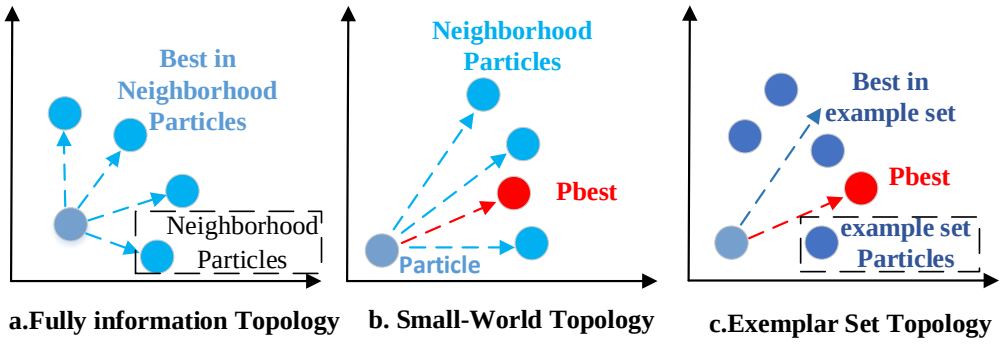


Figure 3. Topology comparison chart.

Under the FIPS topology, the velocity of particle i is influenced by all its neighborhood members with additional weights. Its update equation is

$$v_{i,d}^{t+1} = \omega v_{i,d} + \phi \sum_{j \in N_i} r_{j,d}(p_{j,d}^{best} - x_{i,d}^{t}), \tag{14}$$

where N_i denotes the set of neighbors of particle i, ϕ is an additional weight factor, and $r_{j,d} \sim U(0,1)$ is a random weight. The inertia weight ω is linearly decreased,

$$\omega = \omega_{\max} - (\omega_{\max} - \omega_{\min})\frac{t}{T} \tag{15}$$

where T is the maximum number of iterations, and t is the current iteration index. This design balances global exploration in early iterations and local exploitation in later stages. Under the small-world topology, particles mainly interact with nearby neighbors while introducing long-distance links with a small probability. Its velocity update is

$$v_{i,d}^{t+1} = \omega v_{i,d}^{t} + c_1 r_1 (p_{i,d}^{\text{best}} - x_{i,d}^{t}) + c_2 r_2 (g_{s_i,d}^{t} - x_{i,d}^{t}), \tag{16}$$

where $g_{s_i,d}^{t}$ is the best solution among neighbors, and the learning factors c_1, c_2 vary linearly,

$$c_1 = c_{1,\max} - (c_{1,\max} - c_{1,\min})\frac{t}{T}. \tag{17}$$

$$c_2 = c_{2,\max} - (c_{2,\max} - c_{2,\min})\frac{t}{T}. \tag{18}$$

Under the exemplar-set topology, each particle dynamically selects exemplars from the constructed exemplar set as reference points to enhance diversity and effectively avoid premature convergence. Its velocity update is

$$v_{i,d}^{t+1} = \omega v_{i,d}^{t} + c_1 r_1 (p_{i,d}^{best} - x_{i,d}^{t}) + c_2 r_2 (p_{sbest_i,d}^{best} - x_{i,d}^{t}), \tag{19}$$

where $p_{\text{best},d}^{\text{best}}$ denotes the best individual in the exemplar set. This strategy introduces diverse references, strengthening swarm diversity and improving the ability to escape local optima.

In summary, combining the three topological structures with the dynamic adjustment mechanism of ω, c_1 and c_2 provides Q-learning with a set of structural actions, achieving a balance between exploration and exploitation. The FIPS topology strengthens fully informed aggregation to improve global convergence; the small-world topology balances information transmission and convergence speed, enhancing search robustness; and the exemplar-set topology maintains swarm diversity through diversified reference individuals, reducing the risk of premature convergence.

3.3. Reward Function

In the MSTPSO algorithm, the reward function is designed to quantify the search performance after executing the current strategy, and it serves as the updating signal for the Q-learning module. The design of the reward comprehensively considers both the improvement of particle fitness and the maintenance of population diversity, thereby guiding the search in the desired direction and avoiding premature convergence and local stagnation. Specifically, the reward of a particle in each generation consists of two parts: (1) the fitness improvement, which reflects the enhancement of a particle's solution quality, and (2) the diversity maintenance, which encourages exploration behavior. The reward is defined as follows:

$$r_i^t = w_1 \cdot \Delta f_i^t + w_2 \cdot H_i^t, \tag{20}$$

where $\Delta f_i^t = f_i^{t-1} - f_i^t$ represents the fitness improvement between the current solution and the historical best of particle i, and H_i^t denotes the diversity contribution of the

swarm. In terms of weight setting, this study refers to the related work of Liu et al. [32], in which a reinforcement learning-based PSO parameter control adopts a uniform weighting strategy to balance different optimization objectives. Accordingly, this paper sets $w_1 = w_2 = 0.5$, thereby establishing a stable balance between fitness improvement and diversity maintenance.

The reward is then used to update the Q-value function, following the rule

$$Q(s_i, a_i) \leftarrow (1 - \alpha)Q(s_i, a_i) + \alpha\left[r_i + \gamma \max_a Q(s_i', a)\right], \tag{21}$$

where α is the learning rate and γ is the discount factor.

To balance the rapid response to new information and the long-term stability derived from past experiences, MSTPSO adopts a dual experience replay mechanism. The short-term memory (STM) stores the most recent samples of particle state transitions for rapid adaptation, while the long-term memory (LTM) preserves older and more stable samples to enhance the robustness of strategy learning. Each sample consists of (s_t, a_t, r_t, s_{t+1}) and its corresponding priority. The prioritized update rule is defined as

$$\omega_i = \alpha_{TD}(TD_i) + \alpha_f \Delta f_i^t + \alpha_H \Delta H_i^t + \lambda, \tag{22}$$

where α_{TD}, α_f, and α_H are coefficients corresponding to TD error, fitness improvement, and diversity contribution, respectively, TD_i denotes the temporal-difference (TD) error of the i-th sample, and $\lambda = 10^{-6}$ is a smoothing term to avoid zero priority. The coefficients are set as 0.7, 0.5, and 0.3, respectively.

During the search, STM and LTM are updated at fixed intervals (every 500 evaluations). A replacement strategy with a ratio of 0.7:0.3 is applied, where 70% of new samples replace those in STM, while 30% are promoted to LTM. In addition, the top 10% of high-priority samples in STM are transferred into LTM to preserve valuable experience, ensuring long-term diversity maintenance in strategy learning.

3.4. Stagnation Detection and Hierarchical Response Mechanism

To address the potential issues of premature convergence and search stagnation during the iterative process of Particle Swarm Optimization, this paper proposes a hierarchical response mechanism based on stagnation detection. The mechanism first determines the level at which stagnation occurs (population or individual) and subsequently triggers either a partial restart strategy based on the global best neighborhood or a differential evolution (DE)-based perturbation mechanism, thereby enhancing global exploration capability while maintaining local exploitation performance.

In each iteration, if the improvement of the global-best fitness over 20 consecutive iterations is less than 10^{-6}, or if the velocity magnitude of all particles falls below the lower bound $v_{\min} = 10^{-5}$, the population is deemed stagnant and the restart strategy is executed. If an individual particle's personal best position remains unchanged for 10 consecutive generations, the particle is considered individually stagnant, and the DE-based perturbation mechanism is activated.

3.4.1. Global-Best-Oriented Restart Strategy

When population stagnation is detected, a partial restart strategy centered on the global-best neighborhood is employed: 20% of the particles are randomly selected for position and velocity reinitialization within an adaptive radius around the global best, thereby restoring search diversity. The new position is generated as

$$R_d = \lambda \cdot \max_i \left| x_{i,d} - g_d \right|, \quad \lambda \in (0, 1), \tag{23}$$

$$x_{i,d} \leftarrow g_d + \text{rand}(-R_d, R_d), \tag{24}$$

where g_d denotes the coordinate of the global best in dimension d, and rand$(\cdot)$ represents a uniformly distributed random number. This method redistributes particles around the global best within an adaptive radius, helping the swarm escape stagnation caused by local clustering and velocity decay.

3.4.2. DE-Based Perturbation Mechanism

For particles identified as individually stagnant, a DE-based perturbation mechanism is integrated into the stagnation detection framework to enhance diversity and promote escape from local optima. As shown in Figure 4, the specific operation is as follows:

(1) Global-best region perturbation.

Based on Euclidean distance, the closest G_{near} and farthest G_{far} particles from the global best G_{best} are selected to generate a new candidate,

$$G_{\text{new}} = G_{\text{near}} + F \cdot (G_{\text{far}} - G_{\text{near}}), \tag{25}$$

where F denotes the differential evolution scaling factor, set to F = 0.6 in our experiments. If the fitness of G_{new} is better than that of G_{best}, the latter is replaced by the new one.

(2) Individual-level perturbation.

For each stagnant particle, the nearest and farthest neighbors $p_{\text{near},i}$ and $p_{\text{far},i}$ relative to $p_{\text{best},i}$ are selected,

$$P_{\text{new},i} = P_{\text{near},i} + F \cdot (P_{\text{far},i} - P_{\text{near},i}). \tag{26}$$

If $P_{\text{new},i}$ is better, it updates $p_{\text{best},i}$; otherwise, it replaces the farthest neighbor to increase local diversity.

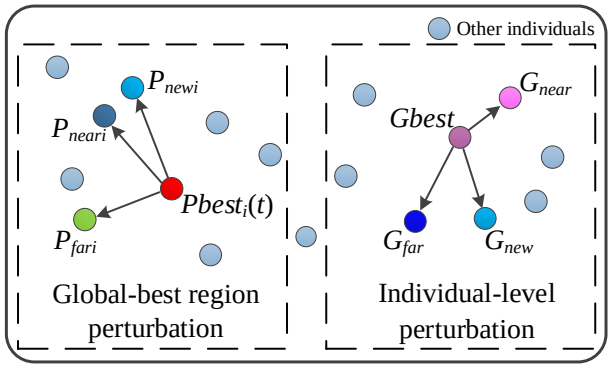


Figure 4. Differential evolutionary perturbations.

3.5. Summary and Complexity Analysis

The MSTPSO algorithm begins with the initialization of particles, PSO parameters, the Q-table, and short-term and long-term experience buffers. In each iteration, particle states are obtained, topology actions are selected through the policy on the Q-table, and particle velocities and positions are updated accordingly. Fitness is then evaluated, personal and global bests are updated, and rewards with transitions are stored into replay buffers. The Q-table is updated regularly with prioritized experience replay. Stagnation detection is performed, and restart or DE-based perturbation strategies are applied when necessary. After completing the iterations, the algorithm returns the global best solution.

As shown in Algorithm 1, the computational complexity of MSTPSO consists of five components: fitness evaluation, Q-learning-based topology selection and particle swarm

update, diversity entropy calculation, restart and DE perturbation, and the dual-layer experience replay mechanism.

Fitness evaluation is performed for N particles and costs O(NC_f), where N is the swarm size and C_f is the cost of a single fitness computation. On CEC benchmark functions, $C_f = O(D)$, with D denoting the problem dimension. In the Q-learning–controlled particle swarm update, each generation each particle selects one of three topologies. For FIPS and small-world topologies, the update is $O(ND)$. When the exemplar-set topology is chosen, it needs to select from msl=[0.1N] exemplars, incurring an additional $O(N^2)$ cost. The diversity entropy module relies on Shannon entropy statistics: interval frequency counting for N particles over D dimensions is $O(ND)$. Restart and DE perturbation are triggered only when stagnation occurs, with amortized complexity $O(ND)$. The dual-layer experience replay comprises a short-term memory (STM, size 2N) and a long-term memory (LTM, size 5N), with storage complexity $O(N)$. Each generation writes N transitions into STM and computes priorities in constant time, giving $O(N)$ per generation. When replay is triggered, B = 3N samples are drawn and the Q-table is updated, costing $O(N)$. Amortized over all generations, this remains linear and negligible compared with the main terms. Considering the entire evolutionary process, the overall time complexity of MSTPSO is $O(N^2D)$.

Algorithm 1 MSTPSO Algorithm

Input: population size N, dimension D, maximum iterations T
Output: global best solution G_{best}

1: Initialize particle positions, velocities, and the Q-table.
2: Create dual-layer replay buffers.
3: Evaluate initial fitness and determine G_{best}.
4: **for** $t = 1$ **to** T **do**
5: **for each** particle **do**
6: Select topology using Q-learning policy.
7: Update $x_{i,d}^t$ and $v_{i,d}^t$ according to the selected topology by (15), (17), and (20).
8: **end for**
9: Evaluate fitness and update personal and global bests.
10: If stagnation is detected, apply perturbation or restart strategy.
11: **end for**
12: Return G_{best}.

4. Numerical Experiments

4.1. Experimental Settings and Comparison Methods

In this experiment, the performance of the proposed MSTPSO algorithm is evaluated using 30 benchmark functions from the CEC2017 [40] test suite. Since function f_2 shows instability in high-dimensional cases, it is excluded from the analysis. Table 1 lists the detailed information of these functions, which are divided into four categories: unimodal functions (f_1, f_3), multimodal functions (f_4–f_{10}), hybrid functions (f_{11}–f_{20}), and composition functions (f_{21}–f_{30}). Extensive numerical experiments are conducted to validate the effectiveness of MSTPSO.

For comparative studies, MSTPSO is benchmarked against nine mainstream PSO variants, including DQNPSO [43], APSO_SAC [44], EPSO [45], KGPSO [46], XPSO [22], MSORL [47], DRA [48], RFO [49], and RLACA [50]. The detailed information of these algorithms is given in Table 2. All functions are independently executed 51 times according to the CEC standard, with the termination condition set as the maximum number of evaluations equal to $D \times 10^4$, where D is the problem dimension.

To comprehensively evaluate the effectiveness of MSTPSO, experiments are conducted on the CEC2017 test suite in 10, 30, 50 and 100 dimensions, and the results are compared

with the above-mentioned algorithms. Tables 3–6 present the mean and standard deviation of the solution errors obtained after 51 runs. In addition, as shown in Table 7, the results of the Wilcoxon signed-rank test demonstrate the statistical significance of the algorithm. The data indicate that MSTPSO demonstrates significant advantages over existing PSO variants, confirming its superior performance.

Table 1. The information of the test suite on CEC2017.

No.	Function Name	Range
f_1	Shifted and Rotated Bent Cigar Function	$[-100, 100]$
f_3	Shifted and Rotated Zakharov Function	$[-100, 100]$
f_4	Shifted and Rotated Rosenbrock's Function	$[-100, 100]$
f_5	Shifted and Rotated Rastrigin's Function	$[-100, 100]$
f_6	Shifted and Rotated Expanded Scaffer's Function	$[-100, 100]$
f_7	Shifted and Rotated Lunacek Bi-Rastrigin's Function	$[-100, 100]$
f_8	Shifted and Rotated Non-Continuous Rastrigin's Function	$[-100, 100]$
f_9	Shifted and Rotated Levy Function	$[-100, 100]$
f_{10}	Shifted and Rotated Schwefel's Function	$[-100, 100]$
f_{11}	Hybrid Function 1 (N = 3)	$[-100, 100]$
f_{12}	Hybrid Function 2 (N = 3)	$[-100, 100]$
f_{13}	Hybrid Function 3 (N = 3)	$[-100, 100]$
f_{14}	Hybrid Function 4 (N = 4)	$[-100, 100]$
f_{15}	Hybrid Function 5 (N = 4)	$[-100, 100]$
f_{16}	Hybrid Function 6 (N = 4)	$[-100, 100]$
f_{17}	Hybrid Function 6 (N = 5)	$[-100, 100]$
f_{18}	Hybrid Function 6 (N = 5)	$[-100, 100]$
f_{19}	Hybrid Function 6 (N = 5)	$[-100, 100]$
f_{20}	Hybrid Function 6 (N = 6)	$[-100, 100]$
f_{21}	Composition Function 1 (N = 3)	$[-100, 100]$
f_{22}	Composition Function 2 (N = 3)	$[-100, 100]$
f_{23}	Composition Function 3 (N = 4)	$[-100, 100]$
f_{24}	Composition Function 4 (N = 4)	$[-100, 100]$
f_{25}	Composition Function 5 (N = 5)	$[-100, 100]$
f_{26}	Composition Function 6 (N = 5)	$[-100, 100]$
f_{27}	Composition Function 7 (N = 6)	$[-100, 100]$
f_{28}	Composition Function 8 (N = 6)	$[-100, 100]$
f_{29}	Composition Function 9 (N = 3)	$[-100, 100]$
f_{30}	Composition Function 10 (N = 3)	$[-100, 100]$

Table 2. Parameter settings for the nine compared algorithms.

Algorithms	Year	Parameters Setting
DQNPSO	2024	$c_1 = 2,\ c_2 = 2,\ \omega = 0.5,\ \gamma = 0.001$
APSO_SAC	2024	$\beta = 1,\ \tau = 0.005,\ \gamma = 0.001$
EPSO	2023	$c_1 = 2,\ c_2 = 2,\ \omega = 0.5$
KGPSO	2020	$c_1 = 2.13 \sim 1.13,\ c_2 = [0.93, 1.75],\ c_3 = [0.6, 1.3],\ \omega = 0.8 \sim 0.35$
XPSO	2023	$\eta = 0.2,\ Stag_{max} = 5,\ p = 0.2,\ \omega = [0.4, 0.9]$
MSORL	2025	$\phi_1 = \phi_2 = 0.4,\ \alpha = 0.4,\ \gamma = 0.8$
DRA	2025	$N = 50,\ BSP = 0.5,\ RP = 0.2$
RFO	2025	$L = 100,\ \beta = 10^{-4}$
RLACA	2023	$c_1 = 2.2,\ c_2 = 1.8,\ \omega = [0, 1]$

Table 3. The comparative results of MSTPSO and advanced algorithms on 10-*D* using the CEC2017 benchmark suite.

Function	10-*D*	MSTPSO	DQNPSO	APSO_SAC	EPSO	KGPSO	XPSO	MSORL	DRA	RFO	RLACA
f1	Mean	6.07×10^{2}	2.15×10^{8}	6.67×10^{8}	1.02×10^{3}	9.45×10^{7}	6.76×10^{6}	7.66×10^{7}	1.20×10^{10}	6.44×10^{5}	$\mathbf{2.77 \times 10^{2}}$
	Std	2.65×10^{-2}	1.09×10^{8}	2.58	1.03×10^{-3}	1.08×10^{6}	4.83×10^{9}	4.09×10^{-2}	5.66×10^{8}	7.01×10^{-5}	2.27×10^{-2}
f3	Mean	**0**	1.06	1.15	6.49×10^{-10}	1.09×10^{2}	1.50×10^{2}	4.07×10^{2}	1.50×10^{4}	1.64×10^{2}	0
	Std	2.10×10^{-1}	8.37×10^{5}	1.16×10^{2}	5.82×10^{-11}	2.54×10^{-13}	3.50×10^{9}	6.31×10^{3}	1.66×10^{3}	1.85×10^{-8}	0
f4	Mean	7.11×10^{-2}	1.95×10^{1}	3.80×10^{1}	3.71	1.14×10^{1}	7.06	2.97×10^{1}	9.12×10^{2}	8.60	**0**
	Std	5.78×10^{-4}	8.35×10^{1}	7.08×10^{-1}	1.42×10^{-3}	4.87	1.13×10^{3}	4.85×10^{-8}	1.29×10^{2}	5.77×10^{-11}	0
f5	Mean	**2.20**	1.45×10^{1}	2.20×10^{1}	3.06×10^{1}	9.83	2.48×10^{1}	4.69×10^{1}	9.43×10^{1}	2.04×10^{1}	1.88×10^{1}
	Std	3.54×10^{-2}	2.39×10^{1}	1.56	1.54	3.27×10^{1}	3.87×10^{1}	2.64×10^{1}	1.50×10^{1}	5.02×10^{-11}	2.48×10^{-7}
f6	Mean	$\mathbf{4.78 \times 10^{-7}}$	5.32×10^{-1}	8.51×10^{-1}	1.25	1.86×10^{-1}	2.11	2.41×10^{1}	5.23×10^{1}	4.78	8.05×10^{-1}
	Std	1.10×10^{-1}	1.35×10^{-1}	1.64×10^{-2}	2.80×10^{-5}	8.37×10^{-9}	1.27×10^{1}	3.48×10^{1}	7.81	2.36×10^{-7}	1.27×10^{-9}
f7	Mean	$\mathbf{1.18 \times 10^{1}}$	1.81×10^{1}	2.39×10^{1}	4.04×10^{1}	2.38×10^{1}	4.83×10^{1}	2.31×10^{1}	1.19×10^{2}	2.99×10^{1}	2.37×10^{1}
	Std	3.35×10^{-4}	7.71×10^{1}	3.03	2.71×10^{-1}	3.32×10^{1}	3.58×10^{1}	3.09	1.22×10^{1}	3.27×10^{-10}	9.71×10^{-14}
f8	Mean	**1.58**	1.16×10^{1}	2.02×10^{1}	1.56×10^{1}	6.90	2.25×10^{1}	1.92×10^{1}	6.06×10^{1}	1.73×10^{1}	1.91×10^{1}
	Std	3.88×10^{-1}	2.54×10^{1}	1.28	1.60×10^{-1}	3.29×10^{1}	2.41×10^{1}	3.10×10^{1}	8.06	6.25×10^{-11}	1.99×10^{-9}
f9	Mean	**0**	1.62×10^{-1}	1.65×10^{1}	5.53×10^{1}	0	3.16	2.85×10^{1}	7.33×10^{2}	2.97×10^{1}	1.78×10^{-2}
	Std	2.96×10^{-2}	1.49×10^{-11}	8.37×10^{-2}	4.37×10^{-1}	9.72×10^{-14}	4.71×10^{2}	1.05×10^{3}	3.00×10^{2}	2.12×10^{-10}	1.38×10^{-14}
f10	Mean	$\mathbf{1.54 \times 10^{2}}$	4.53×10^{2}	5.64×10^{2}	8.77×10^{2}	4.05×10^{2}	8.28×10^{2}	1.86×10^{3}	1.75×10^{3}	6.96×10^{2}	6.63×10^{2}
	Std	7.36×10^{-12}	5.44×10^{2}	6.02×10^{1}	8.53	3.31×10^{2}	6.93×10^{2}	3.45×10^{2}	2.24×10^{2}	3.93×10^{-9}	1.18
f11	Mean	$\mathbf{7.48 \times 10^{-1}}$	2.65×10^{1}	4.66×10^{1}	1.24×10^{1}	9.31	1.63×10^{1}	4.86×10^{1}	1.87×10^{3}	5.65×10^{1}	1.77×10^{1}
	Std	1.76×10^{-4}	2.23×10^{7}	1.83	2.81×10^{-1}	1.52×10^{1}	2.98×10^{6}	3.39×10^{-1}	1.33×10^{4}	2.01×10^{-10}	4.66×10^{-7}
f12	Mean	1.71×10^{4}	6.71×10^{5}	2.75×10^{6}	7.51×10^{3}	6.58×10^{5}	4.65×10^{5}	1.92×10^{4}	2.99×10^{8}	$\mathbf{1.91 \times 10^{3}}$	8.35×10^{3}
	Std	3.63×10^{-1}	4.72×10^{8}	2.38×10^{5}	5.21×10^{-2}	3.19×10^{7}	1.25×10^{9}	1.97×10^{-3}	1.52×10^{8}	5.73×10^{-7}	3.33
f13	Mean	1.89×10^{3}	9.03×10^{3}	1.04×10^{4}	8.01×10^{3}	6.25×10^{3}	7.36×10^{3}	4.36×10^{2}	5.24×10^{4}	2.64×10^{2}	$\mathbf{1.73 \times 10^{2}}$
	Std	2.48×10^{4}	5.32×10^{8}	3.47×10^{3}	2.44×10^{-1}	1.51×10^{7}	3.18×10^{8}	2.59×10^{4}	1.82×10^{7}	6.71×10^{-8}	1.08×10^{-1}
f14	Mean	$\mathbf{4.00 \times 10^{1}}$	1.17×10^{2}	1.93×10^{2}	1.13×10^{3}	5.88×10^{1}	7.48×10^{1}	4.49×10^{1}	2.29×10^{2}	3.03×10^{1}	5.56×10^{1}
	Std	1.59×10^{2}	1.84×10^{8}	7.48×10^{2}	5.55×10^{-1}	6.54×10^{6}	3.65×10^{8}	4.03×10^{4}	2.63×10^{4}	2.90×10^{-9}	2.36×10^{-2}
f15	Mean	2.36×10^{2}	9.14×10^{2}	4.52×10^{2}	1.38×10^{3}	6.83×10^{2}	1.70×10^{2}	7.61×10^{1}	5.62×10^{3}	5.38×10^{1}	$\mathbf{4.18 \times 10^{1}}$
	Std	4.75×10^{4}	1.98×10^{9}	2.35×10^{3}	4.38×10^{-1}	2.58×10^{7}	6.49×10^{8}	2.05×10^{5}	3.41×10^{4}	7.83×10^{-8}	9.40×10^{-1}
f16	Mean	$\mathbf{9.10 \times 10^{-1}}$	7.53×10^{1}	8.93×10^{1}	1.32×10^{2}	1.38×10^{2}	7.68×10^{1}	3.62×10^{2}	4.81×10^{2}	6.40×10^{1}	1.11×10^{2}
	Std	1.99×10^{-2}	5.48×10^{2}	1.22×10^{1}	1.58×10^{1}	2.08×10^{2}	6.45×10^{2}	2.12×10^{2}	1.46×10^{2}	7.40×10^{-8}	1.76×10^{-1}
f17	Mean	$\mathbf{1.88 \times 10^{1}}$	4.73×10^{1}	7.12×10^{1}	4.61×10^{1}	4.60×10^{1}	4.21×10^{1}	1.14×10^{2}	1.32×10^{2}	4.48×10^{1}	5.02×10^{1}
	Std	1.50	1.05×10^{4}	7.55	1.12	1.63×10^{2}	8.43×10^{2}	1.70×10^{2}	7.43×10^{1}	5.22×10^{-9}	1.33×10^{-3}
f18	Mean	2.82×10^{3}	1.12×10^{4}	2.22×10^{4}	1.17×10^{4}	1.73×10^{4}	1.39×10^{4}	1.24×10^{2}	8.39×10^{6}	$\mathbf{8.42 \times 10^{1}}$	2.56×10^{2}
	Std	7.89×10^{4}	1.75×10^{9}	2.98×10^{4}	3.11	1.62×10^{8}	3.15×10^{9}	1.42×10^{-1}	2.82×10^{8}	7.11×10^{-8}	1.22×10^{-2}
f19	Mean	8.71×10^{1}	2.76×10^{3}	1.25×10^{3}	5.81×10^{3}	4.92×10^{3}	1.40×10^{2}	2.36×10^{1}	4.52×10^{3}	$\mathbf{1.98 \times 10^{1}}$	2.46×10^{1}
	Std	4.30×10^{4}	9.72×10^{8}	2.47×10^{3}	4.65×10^{-1}	9.36×10^{7}	1.66×10^{9}	3.00×10^{4}	2.97×10^{7}	6.42×10^{-10}	2.12×10^{-3}
f20	Mean	$\mathbf{1.39 \times 10^{1}}$	1.94×10^{1}	3.87×10^{1}	3.77×10^{1}	7.36×10^{1}	6.04×10^{1}	2.96×10^{2}	2.06×10^{2}	4.56×10^{1}	7.54×10^{1}
	Std	2.38×10^{-1}	8.52×10^{1}	2.41	6.29	4.74×10^{1}	3.47×10^{2}	1.68×10^{2}	6.90×10^{1}	3.83×10^{-8}	1.10×10^{-3}
f21	Mean	$\mathbf{1.55 \times 10^{2}}$	2.10×10^{2}	2.15×10^{2}	2.16×10^{2}	1.99×10^{2}	2.12×10^{2}	2.35×10^{2}	2.40×10^{2}	2.10×10^{2}	1.57×10^{2}
	Std	4.29	2.68×10^{1}	1.25	3.70×10^{-1}	3.07×10^{1}	5.88×10^{1}	6.00×10^{1}	2.85×10^{1}	4.78×10^{-11}	8.87×10^{-5}
f22	Mean	$\mathbf{1.00 \times 10^{2}}$	1.32×10^{2}	1.18×10^{2}	1.03×10^{2}	1.14×10^{2}	1.11×10^{2}	1.43×10^{2}	1.09×10^{3}	1.08×10^{2}	1.02×10^{2}
	Std	8.10×10^{-1}	1.57×10^{1}	6.76×10^{-1}	3.60×10^{-2}	2.98	1.69×10^{2}	1.25×10^{3}	9.00×10^{1}	3.93×10^{-10}	3.96×10^{-13}
f23	Mean	$\mathbf{3.01 \times 10^{2}}$	3.23×10^{2}	3.36×10^{2}	3.22×10^{2}	3.24×10^{2}	3.25×10^{2}	4.13×10^{2}	4.45×10^{2}	3.23×10^{2}	3.20×10^{2}
	Std	2.05×10^{-1}	1.83×10^{1}	6.27×10^{-1}	1.31	6.75	8.53×10^{1}	4.01×10^{2}	2.69×10^{1}	1.15×10^{-10}	1.77×10^{-2}
f24	Mean	3.15×10^{2}	3.38×10^{2}	3.67×10^{2}	3.38×10^{2}	3.21×10^{2}	3.34×10^{2}	$\mathbf{2.82 \times 10^{2}}$	4.39×10^{2}	3.35×10^{2}	3.42×10^{2}
	Std	7.65×10^{-1}	2.87×10^{1}	1.02	8.03×10^{-1}	1.66×10^{1}	8.27×10^{1}	1.33×10^{2}	2.47×10^{1}	9.55×10^{-11}	1.00×10^{-2}
f25	Mean	4.32×10^{2}	4.39×10^{2}	4.50×10^{2}	4.29×10^{2}	4.30×10^{2}	4.33×10^{2}	4.35×10^{2}	1.08×10^{3}	4.34×10^{2}	$\mathbf{4.25 \times 10^{2}}$
	Std	1.04×10^{-1}	2.74×10^{1}	7.72×10^{-1}	7.40×10^{-3}	2.31×10^{-2}	7.22×10^{2}	4.88	6.35×10^{1}	8.83×10^{-11}	1.53×10^{-3}
f26	Mean	3.00×10^{2}	5.16×10^{2}	5.45×10^{2}	3.59×10^{2}	3.66×10^{2}	$\mathbf{2.98 \times 10^{2}}$	6.95×10^{2}	1.50×10^{3}	5.34×10^{2}	3.51×10^{2}
	Std	4.49×10^{-1}	1.02×10^{1}	2.99	5.38×10^{-6}	7.33	6.92×10^{2}	5.69×10^{2}	1.49×10^{2}	3.00×10^{-10}	4.71×10^{-4}
f27	Mean	3.95×10^{2}	4.08×10^{2}	4.14×10^{2}	3.96×10^{2}	4.16×10^{2}	4.01×10^{2}	5.04×10^{2}	4.81×10^{2}	4.07×10^{2}	$\mathbf{3.75 \times 10^{2}}$
	Std	1.81	2.03×10^{1}	9.94×10^{-2}	2.73×10^{-2}	4.05×10^{-2}	8.65×10^{2}	3.11×10^{2}	2.98×10^{1}	5.35×10^{-11}	6.27×10^{-1}
f28	Mean	$\mathbf{3.00 \times 10^{2}}$	5.71×10^{2}	5.21×10^{2}	5.13×10^{2}	5.12×10^{2}	5.04×10^{2}	5.85×10^{2}	9.46×10^{2}	5.09×10^{2}	4.02×10^{2}
	Std	8.68	1.94×10^{1}	6.76×10^{-2}	8.13×10^{-3}	7.84×10^{-13}	1.22×10^{3}	3.79×10^{1}	5.00×10^{1}	6.02×10^{-11}	2.62×10^{1}
f29	Mean	$\mathbf{2.52 \times 10^{2}}$	2.88×10^{2}	3.13×10^{2}	3.08×10^{2}	2.87×10^{2}	2.92×10^{2}	3.83×10^{2}	5.27×10^{2}	2.75×10^{2}	2.83×10^{2}
	Std	4.69×10^{-2}	2.10×10^{3}	9.91	6.91	2.19×10^{2}	5.59×10^{5}	2.91×10^{3}	1.34×10^{2}	3.73×10^{-9}	4.64×10^{-1}
f30	Mean	1.00×10^{5}	4.68×10^{5}	5.62×10^{5}	5.39×10^{5}	2.97×10^{5}	4.01×10^{5}	6.26×10^{4}	8.06×10^{6}	3.53×10^{5}	$\mathbf{2.13 \times 10^{3}}$
	Std	3.29×10^{6}	2.68×10^{8}	1.17×10^{4}	1.20×10^{2}	1.12×10^{7}	7.71×10^{8}	2.40×10^{7}	1.12×10^{7}	3.01×10^{-9}	1.16×10^{5}

Table 4. The comparative results of MSTPSO and advanced algorithms on 30-*D* using the CEC2017 benchmark suite.

Function	30-*D*	MSTPSO	DQNPSO	APSO_SAC	EPSO	KGPSO	XPSO	MSORL	DRA	RFO	RLACA
f1	Mean	1.42×10^{3}	7.01×10^{9}	6.54×10^{9}	3.80×10^{3}	1.06×10^{9}	1.34×10^{9}	1.33×10^{10}	6.40×10^{10}	2.69×10^{10}	$\mathbf{1.66 \times 10^{-2}}$
	Std	1.42×10^{-1}	1.47×10^{8}	2.19×10^{3}	4.46×10^{-3}	4.79×10^{3}	6.91×10^{9}	6.85×10^{-3}	4.06×10^{7}	4.71×10^{-2}	8.71×10^{-8}
f3	Mean	1.58×10^{3}	6.79×10^{3}	4.57×10^{4}	5.59×10^{3}	6.64×10^{2}	8.48×10^{3}	3.03×10^{4}	8.50×10^{4}	6.48×10^{4}	**0**
	Std	3.19×10^{7}	6.95×10^{13}	2.89×10^{3}	6.29	5.16×10^{-4}	2.05×10^{13}	2.80×10^{-7}	5.61×10^{3}	9.33×10^{-7}	7.15×10^{-14}
f4	Mean	8.66×10^{1}	1.06×10^{3}	7.13×10^{2}	7.89×10^{1}	1.72×10^{2}	2.59×10^{2}	2.25×10^{3}	1.63×10^{4}	4.18×10^{3}	$\mathbf{5.02 \times 10^{1}}$
	Std	2.46×10^{2}	5.39×10^{1}	1.13	2.34×10^{-3}	6.55	3.01×10^{4}	1.51×10^{-7}	7.43×10^{1}	1.07×10^{-8}	6.33×10^{-1}
f5	Mean	$\mathbf{1.23 \times 10^{1}}$	1.22×10^{2}	1.66×10^{2}	1.47×10^{2}	4.67×10^{1}	1.87×10^{2}	2.43×10^{2}	4.49×10^{2}	2.55×10^{2}	1.11×10^{2}
	Std	6.84×10^{-1}	2.76	2.82	1.05	6.62×10^{1}	5.66×10^{1}	5.68×10^{1}	1.13×10^{1}	1.43×10^{-9}	1.02×10^{-4}
f6	Mean	$\mathbf{1.88 \times 10^{-3}}$	1.18×10^{1}	1.68×10^{1}	2.63×10^{1}	1.01	1.60×10^{1}	5.69×10^{1}	9.21×10^{1}	4.86×10^{1}	8.75
	Std	1.12×10^{-6}	4.69×10^{-1}	1.09×10^{-1}	1.78×10^{-4}	1.46×10^{-8}	9.76	2.24×10^{1}	3.60	5.00×10^{-10}	7.64×10^{-8}
f7	Mean	$\mathbf{3.89 \times 10^{1}}$	2.18×10^{2}	2.32×10^{2}	2.35×10^{2}	1.02×10^{2}	3.10×10^{2}	3.17×10^{2}	7.85×10^{2}	5.63×10^{2}	1.29×10^{2}
	Std	6.04	3.46	1.47×10^{1}	2.12	8.00×10^{1}	9.00×10^{1}	2.13×10^{-1}	1.19×10^{1}	5.94×10^{-9}	1.48×10^{-13}
f8	Mean	$\mathbf{1.14 \times 10^{1}}$	1.21×10^{2}	1.38×10^{2}	1.25×10^{2}	4.01×10^{1}	1.83×10^{2}	1.77×10^{2}	3.65×10^{2}	2.17×10^{2}	1.08×10^{2}
	Std	3.75	2.02	3.68	1.83	6.54×10^{1}	5.13×10^{1}	5.92×10^{1}	7.89	1.35×10^{-9}	3.60×10^{-3}
f9	Mean	$\mathbf{6.91 \times 10^{-2}}$	1.64×10^{3}	2.16×10^{3}	2.98×10^{3}	6.47×10^{1}	6.70×10^{2}	2.65×10^{3}	1.04×10^{4}	4.14×10^{3}	2.23×10^{2}
	Std	4.64×10^{-6}	3.32×10^{3}	5.82×10^{2}	3.70×10^{1}	5.22×10^{1}	2.07×10^{3}	1.28×10^{3}	1.31×10^{3}	1.84×10^{-7}	1.52×10^{-2}
f10	Mean	$\mathbf{1.77 \times 10^{3}}$	3.22×10^{3}	3.69×10^{3}	3.63×10^{3}	2.30×10^{3}	6.67×10^{3}	6.78×10^{3}	8.25×10^{3}	5.08×10^{3}	3.37×10^{3}
	Std	1.20	1.09×10^{3}	7.65×10^{2}	4.72	1.74×10^{3}	1.10×10^{3}	8.34×10^{2}	2.07×10^{2}	3.76×10^{-8}	1.81×10^{1}
f11	Mean	$\mathbf{2.77 \times 10^{1}}$	3.80×10^{2}	7.50×10^{2}	1.21×10^{2}	9.27×10^{1}	3.62×10^{2}	6.48×10^{2}	7.86×10^{3}	1.36×10^{3}	1.25×10^{2}
	Std	1.91×10^{1}	4.42×10^{4}	1.01	6.75×10^{-5}	3.23	1.29×10^{8}	1.15×10^{-5}	1.05×10^{4}	1.73×10^{-8}	1.76×10^{-1}
f12	Mean	$\mathbf{2.33 \times 10^{4}}$	3.50×10^{8}	2.70×10^{8}	1.52×10^{5}	6.60×10^{7}	1.11×10^{8}	9.22×10^{8}	1.63×10^{10}	9.87×10^{8}	1.55×10^{5}
	Std	1.02×10^{5}	4.09×10^{7}	1.02×10^{6}	4.16×10^{1}	5.21×10^{6}	2.57×10^{9}	2.15×10^{-3}	6.72×10^{7}	5.46×10^{-3}	1.02×10^{4}
f13	Mean	$\mathbf{5.45 \times 10^{3}}$	1.64×10^{8}	5.68×10^{8}	1.19×10^{4}	4.71×10^{7}	1.21×10^{7}	2.75×10^{7}	9.49×10^{9}	1.63×10^{5}	1.39×10^{4}
	Std	1.37×10^{1}	1.67×10^{8}	4.36×10^{1}	2.97×10^{-1}	3.08×10^{1}	6.75×10^{9}	7.35×10^{-1}	3.74×10^{8}	2.30×10^{-5}	2.99×10^{-1}
f14	Mean	2.60×10^{3}	2.65×10^{4}	6.00×10^{4}	1.16×10^{4}	9.28×10^{3}	2.73×10^{4}	3.21×10^{3}	3.80×10^{6}	$\mathbf{3.11 \times 10^{2}}$	$\mathbf{3.11 \times 10^{2}}$
	Std	2.31	3.29×10^{8}	4.91×10^{4}	5.56×10^{-1}	6.88×10^{6}	4.60×10^{8}	3.19×10^{-8}	5.38×10^{6}	6.18×10^{-8}	7.95×10^{-2}
f15	Mean	3.15×10^{3}	6.16×10^{4}	6.42×10^{4}	$\mathbf{2.35 \times 10^{3}}$	6.17×10^{3}	5.24×10^{5}	1.31×10^{4}	1.02×10^{9}	1.91×10^{4}	3.43×10^{3}
	Std	8.83	8.60×10^{7}	2.92×10^{4}	2.42×10^{-3}	6.41×10^{5}	2.56×10^{9}	4.04×10^{-7}	3.43×10^{7}	5.06×10^{-6}	1.08×10^{-2}
f16	Mean	$\mathbf{2.12 \times 10^{2}}$	8.88×10^{2}	1.34×10^{3}	1.14×10^{3}	6.14×10^{2}	1.39×10^{3}	1.86×10^{3}	4.43×10^{3}	1.24×10^{3}	9.41×10^{2}
	Std	1.95×10^{1}	4.61×10^{2}	1.09×10^{1}	1.86×10^{1}	3.70×10^{2}	2.89×10^{3}	4.03×10^{2}	3.81×10^{2}	8.63×10^{-9}	1.45
f17	Mean	$\mathbf{3.05 \times 10^{1}}$	4.64×10^{2}	7.04×10^{2}	4.14×10^{2}	2.65×10^{2}	3.18×10^{2}	8.90×10^{2}	3.64×10^{3}	4.95×10^{2}	4.69×10^{2}
	Std	1.14×10^{-2}	9.03×10^{3}	2.00×10^{1}	4.40×10^{1}	1.93×10^{2}	3.27×10^{4}	2.31×10^{2}	3.34×10^{2}	3.99×10^{-9}	2.13
f18	Mean	8.13×10^{4}	2.42×10^{5}	1.57×10^{6}	1.26×10^{5}	2.35×10^{5}	8.33×10^{5}	1.35×10^{5}	3.51×10^{7}	4.53×10^{4}	$\mathbf{2.38 \times 10^{4}}$
	Std	4.28×10^{1}	1.02×10^{9}	2.89×10^{5}	5.50	9.20×10^{7}	1.98×10^{9}	4.79×10^{-7}	2.79×10^{7}	3.24×10^{-6}	1.85×10^{4}
f19	Mean	5.31×10^{3}	1.94×10^{6}	6.53×10^{7}	$\mathbf{4.85 \times 10^{3}}$	9.62×10^{3}	1.02×10^{6}	3.11×10^{5}	9.23×10^{8}	2.28×10^{5}	1.23×10^{3}
	Std	5.07×10^{-2}	9.31×10^{6}	2.82×10^{2}	5.34×10^{-4}	8.94×10^{5}	1.48×10^{9}	5.08×10^{1}	3.37×10^{7}	1.70×10^{-5}	2.96×10^{-2}
f20	Mean	$\mathbf{1.38 \times 10^{2}}$	3.00×10^{2}	4.91×10^{2}	4.02×10^{2}	2.32×10^{2}	4.01×10^{2}	7.35×10^{2}	1.03×10^{3}	4.18×10^{2}	5.16×10^{2}
	Std	2.78×10^{-2}	2.35×10^{2}	3.69×10^{1}	1.84×10^{1}	1.62×10^{2}	5.72×10^{2}	3.99×10^{2}	1.32×10^{2}	5.76×10^{-9}	1.36
f21	Mean	$\mathbf{2.12 \times 10^{2}}$	3.27×10^{2}	3.55×10^{2}	3.16×10^{2}	2.58×10^{2}	3.78×10^{2}	4.52×10^{2}	6.69×10^{2}	4.48×10^{2}	3.11×10^{2}
	Std	5.96×10^{-1}	1.03	4.33	2.75	6.82×10^{1}	6.36×10^{1}	7.99×10^{1}	1.50×10^{1}	1.32×10^{-9}	1.85×10^{-2}
f22	Mean	$\mathbf{1.00 \times 10^{2}}$	2.55×10^{3}	3.76×10^{3}	3.57×10^{2}	8.81×10^{2}	6.13×10^{2}	3.45×10^{3}	7.71×10^{3}	4.58×10^{3}	1.47×10^{3}
	Std	1.34×10^{-3}	6.98×10^{2}	3.59×10^{2}	1.02×10^{-8}	7.62×10^{1}	1.03×10^{3}	2.82×10^{3}	1.40×10^{2}	2.64×10^{-8}	8.03
f23	Mean	$\mathbf{3.48 \times 10^{2}}$	5.86×10^{2}	6.79×10^{2}	4.93×10^{2}	5.69×10^{2}	5.43×10^{2}	9.94×10^{2}	1.27×10^{3}	7.15×10^{2}	5.29×10^{2}
	Std	2.82	6.21×10^{-1}	2.24	2.23	6.94	2.84×10^{2}	6.45×10^{2}	5.20×10^{1}	1.41×10^{-9}	1.28×10^{-1}
f24	Mean	$\mathbf{4.23 \times 10^{2}}$	7.15×10^{2}	7.31×10^{2}	5.60×10^{2}	6.58×10^{2}	6.05×10^{2}	9.45×10^{2}	1.40×10^{3}	7.64×10^{2}	6.23×10^{2}
	Std	1.33	1.49×10^{1}	6.48	3.68	2.17×10^{1}	2.14×10^{2}	2.35×10^{1}	3.12×10^{1}	1.20×10^{-9}	6.97×10^{-2}
f25	Mean	3.88×10^{2}	6.64×10^{2}	6.70×10^{2}	3.98×10^{2}	$\mathbf{3.99 \times 10^{2}}$	4.95×10^{2}	8.71×10^{2}	3.04×10^{3}	1.49×10^{3}	3.80×10^{2}
	Std	8.40	3.67×10^{1}	8.41×10^{-1}	4.81×10^{-13}	5.13×10^{-13}	2.24×10^{3}	9.03×10^{-13}	2.14×10^{1}	2.98×10^{-9}	9.24×10^{-2}
f26	Mean	$\mathbf{9.17 \times 10^{2}}$	3.47×10^{3}	3.47×10^{3}	2.14×10^{3}	1.65×10^{3}	2.13×10^{3}	5.25×10^{3}	9.73×10^{3}	5.13×10^{3}	1.87×10^{3}
	Std	1.00×10^{-2}	2.80×10^{1}	1.44×10^{1}	1.10×10^{1}	7.20×10^{1}	2.39×10^{3}	9.83×10^{1}	7.23×10^{1}	1.42×10^{-8}	6.39×10^{-1}
f27	Mean	5.17×10^{2}	6.04×10^{2}	6.04×10^{2}	5.30×10^{2}	5.88×10^{2}	5.64×10^{2}	1.09×10^{3}	1.71×10^{3}	6.55×10^{2}	$\mathbf{5.00 \times 10^{2}}$
	Std	6.22	5.28×10^{-1}	3.68×10^{-1}	4.83×10^{-5}	1.81×10^{-1}	1.49×10^{3}	4.55×10^{2}	7.85×10^{1}	5.51×10^{-10}	9.05×10^{1}
f28	Mean	3.57×10^{2}	1.19×10^{3}	1.05×10^{3}	4.01×10^{2}	4.97×10^{2}	5.74×10^{2}	1.35×10^{3}	5.23×10^{3}	2.03×10^{3}	$\mathbf{3.66 \times 10^{2}}$
	Std	2.84×10^{1}	5.91×10^{-4}	4.59×10^{-1}	2.24×10^{-3}	1.70×10^{1}	5.63×10^{3}	4.56×10^{-12}	7.74	2.88×10^{-9}	1.58×10^{-1}
f29	Mean	$\mathbf{4.43 \times 10^{2}}$	1.04×10^{3}	1.16×10^{3}	8.33×10^{2}	6.55×10^{2}	1.07×10^{3}	2.32×10^{3}	6.62×10^{3}	1.40×10^{3}	7.72×10^{2}
	Std	2.54×10^{-1}	3.62×10^{4}	6.94	3.86×10^{1}	1.20×10^{2}	6.77×10^{5}	1.37×10^{2}	4.12×10^{2}	5.56×10^{-9}	2.41×10^{1}
f30	Mean	$\mathbf{4.16 \times 10^{3}}$	4.87×10^{6}	1.85×10^{6}	4.96×10^{3}	9.40×10^{4}	4.27×10^{6}	1.32×10^{7}	1.81×10^{9}	2.16×10^{6}	1.17×10^{4}
	Std	1.58×10^{3}	1.19×10^{6}	3.77×10^{3}	3.21×10^{-2}	1.67×10^{5}	9.84×10^{8}	8.78	6.46×10^{7}	4.54×10^{-5}	2.85×10^{5}

Table 5. The comparative results of MSTPSO and advanced algorithms on 50-*D* using the CEC2017 benchmark suite.

Function	50-*D*	MSTPSO	DQNPSO	APSO_SAC	EPSO	KGPSO	XPSO	MSORL	DRA	RFO	RLACA
f1	Mean	1.97×10^{3}	3.59×10^{10}	2.14×10^{10}	$\mathbf{1.62 \times 10^{3}}$	3.64×10^{9}	6.26×10^{9}	4.81×10^{10}	1.21×10^{11}	8.59×10^{10}	3.43×10^{3}
	Std	2.27×10^{-3}	2.97×10^{8}	1.75×10^{6}	1.02×10^{-4}	4.95×10^{1}	1.04×10^{10}	8.56×10^{-3}	4.19×10^{6}	8.60×10^{-2}	4.35×10^{-5}
f3	Mean	1.60×10^{4}	3.14×10^{4}	1.35×10^{5}	3.25×10^{4}	3.69×10^{3}	2.47×10^{4}	9.15×10^{4}	1.81×10^{5}	1.67×10^{5}	**0**
	Std	1.11×10^{3}	7.63×10^{14}	8.05×10^{3}	8.24	2.00×10^{7}	6.11×10^{14}	1.09×10^{5}	2.06×10^{5}	2.27×10^{-6}	7.01×10^{-13}
f4	Mean	1.84×10^{2}	4.37×10^{3}	2.85×10^{3}	1.16×10^{2}	3.56×10^{2}	7.01×10^{2}	9.18×10^{3}	4.37×10^{4}	1.78×10^{4}	$\mathbf{8.65 \times 10^{1}}$
	Std	1.59×10^{2}	1.54×10^{2}	2.56	7.57×10^{-4}	2.19×10^{1}	1.59×10^{4}	2.86×10^{-10}	6.56	2.77×10^{-8}	8.86×10^{-1}
f5	Mean	$\mathbf{2.45 \times 10^{1}}$	2.77×10^{2}	3.40×10^{2}	2.70×10^{2}	8.98×10^{1}	3.91×10^{2}	4.55×10^{2}	7.29×10^{2}	5.39×10^{2}	2.48×10^{2}
	Std	1.06×10^{-2}	1.93×10^{-1}	2.86	4.56×10^{-3}	6.03×10^{1}	6.10×10^{1}	5.65×10^{1}	4.13	2.17×10^{-9}	4.24×10^{-9}
f6	Mean	$\mathbf{3.77 \times 10^{-2}}$	3.09×10^{1}	3.52×10^{1}	4.04×10^{1}	1.88	2.43×10^{1}	7.02×10^{1}	1.05×10^{2}	6.90×10^{1}	2.11×10^{1}
	Std	6.42×10^{-2}	1.11×10^{-1}	2.71×10^{-1}	8.75×10^{-5}	3.08×10^{-8}	8.04	1.83×10^{1}	2.87	5.57×10^{-10}	1.23×10^{-7}
f7	Mean	$\mathbf{6.70 \times 10^{1}}$	8.42×10^{2}	7.46×10^{2}	5.36×10^{2}	1.48×10^{2}	6.55×10^{2}	8.06×10^{2}	1.44×10^{3}	1.32×10^{3}	3.25×10^{2}
	Std	7.84	2.56×10^{1}	5.71	4.52	1.06×10^{2}	1.31×10^{2}	2.98×10^{-10}	8.97	8.97×10^{-9}	1.77×10^{-13}
f8	Mean	$\mathbf{2.65 \times 10^{1}}$	2.90×10^{2}	3.57×10^{2}	2.78×10^{2}	9.09×10^{1}	3.87×10^{2}	4.71×10^{2}	7.48×10^{2}	5.57×10^{2}	2.41×10^{2}
	Std	1.32×10^{-1}	3.35×10^{-1}	1.25	1.43	6.28×10^{1}	6.35×10^{1}	5.53×10^{1}	8.06	2.14×10^{-9}	1.14×10^{-2}
f9	Mean	$\mathbf{7.86 \times 10^{-1}}$	7.51×10^{3}	1.20×10^{4}	9.69×10^{3}	8.36×10^{2}	2.74×10^{3}	1.42×10^{4}	3.78×10^{4}	2.07×10^{4}	2.19×10^{3}
	Std	7.19×10^{-10}	8.59×10^{3}	7.00×10^{3}	3.03×10^{1}	7.03×10^{2}	7.86×10^{3}	4.15×10^{3}	2.41×10^{3}	9.71×10^{-7}	7.20×10^{-2}
f10	Mean	$\mathbf{3.35 \times 10^{3}}$	6.33×10^{3}	7.09×10^{3}	6.85×10^{3}	4.32×10^{3}	1.25×10^{4}	1.20×10^{4}	1.41×10^{4}	1.06×10^{4}	6.55×10^{3}
	Std	1.00	1.51×10^{3}	1.28×10^{3}	1.22×10^{2}	2.25×10^{3}	1.42×10^{3}	1.46×10^{3}	2.69×10^{2}	5.34×10^{-8}	4.04×10^{1}
f11	Mean	$\mathbf{6.72 \times 10^{1}}$	8.17×10^{2}	1.83×10^{3}	2.03×10^{2}	2.01×10^{2}	9.85×10^{2}	4.49×10^{3}	2.62×10^{4}	1.16×10^{4}	2.75×10^{2}
	Std	1.07×10^{2}	1.91×10^{8}	7.83×10^{1}	2.98×10^{-5}	1.52	2.29×10^{9}	3.38×10^{-10}	4.86×10^{2}	1.10×10^{-7}	5.25×10^{-2}
f12	Mean	$\mathbf{1.82 \times 10^{5}}$	6.56×10^{9}	8.12×10^{9}	9.43×10^{5}	2.46×10^{9}	1.22×10^{9}	1.41×10^{10}	1.01×10^{11}	2.40×10^{10}	2.21×10^{6}
	Std	4.44×10^{5}	9.40×10^{7}	2.34×10^{7}	5.74×10^{1}	5.54×10^{7}	1.19×10^{10}	5.73×10^{-3}	6.81×10^{7}	4.94×10^{-2}	3.08×10^{3}
f13	Mean	$\mathbf{2.24 \times 10^{3}}$	1.77×10^{9}	2.00×10^{9}	4.13×10^{3}	7.36×10^{8}	1.95×10^{8}	2.55×10^{9}	6.29×10^{10}	2.75×10^{9}	1.74×10^{4}
	Std	1.78×10^{2}	1.43×10^{8}	3.06×10^{7}	8.48×10^{-2}	2.34	9.68×10^{9}	2.59	1.58×10^{8}	1.65×10^{-2}	2.97×10^{-1}
f14	Mean	1.68×10^{4}	2.42×10^{5}	1.60×10^{6}	4.24×10^{4}	2.01×10^{5}	4.01×10^{5}	1.04×10^{6}	9.07×10^{7}	5.39×10^{4}	$\mathbf{3.68 \times 10^{3}}$
	Std	1.19×10^{6}	2.17×10^{8}	2.04×10^{5}	1.91	8.98×10^{6}	8.12×10^{8}	2.73×10^{-7}	7.65×10^{6}	2.47×10^{-6}	1.62×10^{1}
f15	Mean	$\mathbf{1.06 \times 10^{3}}$	7.50×10^{7}	1.25×10^{8}	8.53×10^{3}	1.41×10^{6}	2.15×10^{7}	5.53×10^{7}	1.25×10^{10}	7.00×10^{5}	9.03×10^{3}
	Std	9.52×10^{1}	1.68×10^{8}	1.05×10^{1}	8.85×10^{-5}	1.61×10^{1}	2.87×10^{9}	3.72×10^{-4}	1.14×10^{7}	9.66×10^{-5}	7.55×10^{-2}
f16	Mean	$\mathbf{5.49 \times 10^{2}}$	2.02×10^{3}	2.26×10^{3}	1.92×10^{3}	1.05×10^{3}	2.74×10^{3}	3.83×10^{3}	9.23×10^{3}	3.14×10^{3}	1.74×10^{3}
	Std	2.96	2.36×10^{2}	6.42×10^{1}	4.72×10^{1}	2.28×10^{2}	1.99×10^{3}	4.46×10^{2}	1.98×10^{2}	1.60×10^{-8}	2.33
f17	Mean	$\mathbf{3.02 \times 10^{2}}$	1.73×10^{3}	2.10×10^{3}	1.49×10^{3}	8.10×10^{2}	1.99×10^{3}	2.00×10^{3}	2.45×10^{4}	1.81×10^{3}	1.28×10^{3}
	Std	5.47×10^{-7}	4.15×10^{6}	4.11×10^{1}	3.89×10^{1}	6.12×10^{2}	7.09×10^{4}	1.91×10^{2}	1.43×10^{2}	1.14×10^{-8}	8.05×10^{-1}
f18	Mean	7.53×10^{4}	1.97×10^{6}	5.41×10^{6}	2.00×10^{5}	8.94×10^{5}	3.65×10^{6}	6.00×10^{6}	2.78×10^{8}	4.55×10^{5}	$\mathbf{7.15 \times 10^{4}}$
	Std	3.60×10^{4}	8.49×10^{8}	9.47×10^{5}	2.32×10^{1}	4.09×10^{7}	2.27×10^{9}	2.87×10^{-4}	7.53×10^{6}	1.01×10^{-5}	3.60×10^{2}
f19	Mean	8.05×10^{3}	1.09×10^{7}	5.09×10^{7}	1.62×10^{4}	1.53×10^{5}	1.15×10^{7}	9.31×10^{6}	4.97×10^{9}	2.09×10^{7}	$\mathbf{5.33 \times 10^{3}}$
	Std	2.66×10^{-2}	1.32×10^{7}	1.56×10^{1}	1.59×10^{-3}	8.18×10^{-1}	1.06×10^{9}	3.00×10^{-2}	1.66×10^{7}	3.91×10^{-4}	3.49×10^{-2}
f20	Mean	$\mathbf{1.34 \times 10^{2}}$	1.02×10^{3}	1.25×10^{3}	1.04×10^{3}	4.18×10^{2}	1.38×10^{3}	1.34×10^{3}	2.53×10^{3}	1.08×10^{3}	1.12×10^{3}
	Std	7.74×10^{-5}	6.70×10^{2}	2.26×10^{2}	6.07×10^{1}	7.06×10^{2}	6.61×10^{2}	6.27×10^{2}	1.11×10^{2}	8.26×10^{-9}	1.15
f21	Mean	$\mathbf{2.26 \times 10^{2}}$	5.24×10^{2}	5.59×10^{2}	4.21×10^{2}	3.33×10^{2}	5.78×10^{2}	7.81×10^{2}	1.19×10^{3}	7.91×10^{2}	4.43×10^{2}
	Std	8.27×10^{-5}	4.40×10^{-1}	8.11	5.03	4.83×10^{1}	6.63×10^{1}	1.70×10^{2}	1.76×10^{1}	2.03×10^{-9}	3.17×10^{-2}
f22	Mean	$\mathbf{1.70 \times 10^{2}}$	6.63×10^{3}	7.60×10^{3}	7.92×10^{3}	3.84×10^{3}	9.63×10^{3}	1.30×10^{4}	1.50×10^{4}	1.07×10^{4}	6.48×10^{3}
	Std	2.13×10^{-3}	1.33×10^{3}	1.03×10^{3}	6.09×10^{1}	1.27×10^{3}	1.58×10^{3}	1.34×10^{3}	2.86×10^{2}	4.87×10^{-8}	2.37×10^{1}
f23	Mean	$\mathbf{4.39 \times 10^{2}}$	1.08×10^{3}	1.13×10^{3}	7.24×10^{2}	9.82×10^{2}	8.31×10^{2}	1.81×10^{3}	2.40×10^{3}	1.36×10^{3}	8.00×10^{2}
	Std	1.46×10^{-2}	1.55	9.60×10^{-1}	5.31	5.33	4.34×10^{2}	8.50×10^{2}	5.80×10^{1}	2.43×10^{-9}	1.79×10^{-1}
f24	Mean	$\mathbf{5.02 \times 10^{2}}$	1.20×10^{3}	1.17×10^{3}	8.01×10^{2}	1.06×10^{3}	8.87×10^{2}	1.68×10^{3}	2.41×10^{3}	1.39×10^{3}	9.54×10^{2}
	Std	5.71×10^{-10}	4.97	2.72	3.31	3.88×10^{1}	3.72×10^{2}	1.71×10^{1}	3.55×10^{1}	1.73×10^{-9}	1.89×10^{-1}
f25	Mean	5.65×10^{2}	3.11×10^{3}	1.79×10^{3}	5.71×10^{2}	5.74×10^{2}	1.02×10^{3}	5.28×10^{3}	1.45×10^{4}	9.42×10^{3}	$\mathbf{4.65 \times 10^{2}}$
	Std	1.47×10^{2}	4.30×10^{1}	2.46	2.81×10^{-4}	1.52×10^{1}	5.20×10^{3}	9.92×10^{-11}	3.16	1.33×10^{-8}	8.57×10^{-1}
f26	Mean	$\mathbf{1.24 \times 10^{3}}$	7.65×10^{3}	7.98×10^{3}	3.76×10^{3}	2.62×10^{3}	3.79×10^{3}	1.08×10^{4}	1.55×10^{4}	1.23×10^{4}	2.97×10^{3}
	Std	4.43×10^{-4}	3.85×10^{1}	2.06×10^{1}	2.91×10^{1}	1.10×10^{2}	3.43×10^{3}	2.91	1.38×10^{1}	2.03×10^{-8}	1.39×10^{-1}
f27	Mean	5.68×10^{2}	1.23×10^{3}	1.13×10^{3}	7.79×10^{2}	1.01×10^{3}	7.86×10^{2}	3.33×10^{3}	4.11×10^{3}	1.47×10^{3}	$\mathbf{5.00 \times 10^{2}}$
	Std	2.07×10^{1}	3.63	1.07	5.06×10^{-5}	2.86×10^{-1}	4.92×10^{3}	9.51×10^{2}	1.16×10^{2}	1.49×10^{-9}	1.44×10^{2}
f28	Mean	$\mathbf{4.70 \times 10^{2}}$	5.80×10^{3}	4.40×10^{3}	5.14×10^{2}	9.21×10^{2}	7.81×10^{2}	4.74×10^{3}	1.24×10^{4}	7.32×10^{3}	4.96×10^{2}
	Std	1.16×10^{2}	9.28	9.75×10^{-1}	7.34×10^{-4}	1.30×10^{1}	4.41×10^{3}	4.42×10^{-11}	5.36	7.14×10^{-9}	1.19×10^{2}
f29	Mean	$\mathbf{4.70 \times 10^{2}}$	2.51×10^{3}	2.61×10^{3}	1.47×10^{3}	1.03×10^{3}	2.34×10^{3}	5.75×10^{3}	1.83×10^{5}	4.63×10^{3}	1.24×10^{3}
	Std	5.87	1.14×10^{4}	5.47	2.10×10^{1}	1.77×10^{1}	1.16×10^{7}	1.34×10^{1}	4.28×10^{3}	1.48×10^{-8}	1.88×10^{1}
f30	Mean	1.22×10^{6}	1.14×10^{8}	3.55×10^{7}	1.11×10^{6}	4.66×10^{6}	1.50×10^{8}	3.46×10^{8}	9.48×10^{9}	2.21×10^{8}	$\mathbf{8.17 \times 10^{4}}$
	Std	1.45×10^{5}	4.47×10^{6}	1.31×10^{5}	2.33	2.20×10^{6}	5.37×10^{9}	7.35×10^{2}	3.78×10^{7}	1.59×10^{-3}	1.32×10^{6}

Table 6. The comparative results of MSTPSO and advanced algorithms on 100-*D* using the CEC2017 benchmark suite.

	100-*D*	MSTPSO	DQNPSO	APSO_SAC	EPSO	KGPSO	XPSO	MSORL	DRA	RFO	RLACA
f1	Mean	$\mathbf{4.16 \times 10^{3}}$	1.77×10^{11}	9.16×10^{10}	5.48×10^{3}	1.38×10^{10}	6.26×10^{9}	4.81×10^{10}	2.81×10^{11}	2.47×10^{11}	1.72×10^{4}
	Std	2.54×10^{-3}	6.49×10^{8}	7.57×10^{7}	3.23×10^{-2}	4.06×10^{7}	1.04×10^{10}	8.56×10^{-3}	6.35×10^{5}	1.55×10^{-1}	5.74×10^{-3}
f3	Mean	1.40×10^{5}	1.93×10^{5}	6.58×10^{5}	1.61×10^{5}	3.18×10^{4}	2.47×10^{4}	9.15×10^{4}	3.47×10^{5}	4.17×10^{5}	$\mathbf{2.56 \times 10^{-1}}$
	Std	6.65×10^{11}	2.66×10^{16}	4.44×10^{4}	1.52×10^{1}	1.42×10^{12}	6.11×10^{14}	1.09×10^{5}	3.04×10^{5}	5.09×10^{-6}	3.69×10^{-6}
f4	Mean	2.97×10^{2}	3.15×10^{4}	1.35×10^{4}	2.71×10^{2}	1.47×10^{3}	7.01×10^{2}	9.18×10^{3}	1.24×10^{5}	7.04×10^{4}	$\mathbf{2.37 \times 10^{2}}$
	Std	1.84×10^{2}	3.06×10^{1}	1.72×10^{1}	1.10×10^{-3}	9.07	1.59×10^{4}	2.86×10^{-10}	4.50	8.30×10^{-8}	6.23×10^{-1}
f5	Mean	$\mathbf{6.77 \times 10^{1}}$	9.76×10^{2}	9.62×10^{2}	7.09×10^{2}	2.48×10^{2}	3.91×10^{2}	4.55×10^{2}	1.67×10^{3}	1.44×10^{3}	6.66×10^{2}
	Std	1.56×10^{-5}	1.58	7.19	1.81	5.39×10^{1}	6.10×10^{1}	5.65×10^{1}	4.10	3.16×10^{-9}	8.89×10^{-10}
f6	Mean	$\mathbf{3.53 \times 10^{-1}}$	6.06×10^{1}	5.87×10^{1}	5.08×10^{1}	3.84	2.43×10^{1}	7.02×10^{1}	1.16×10^{2}	9.22×10^{1}	4.43×10^{1}
	Std	3.32×10^{-9}	1.53×10^{-2}	3.43×10^{-1}	8.74×10^{-5}	8.16×10^{-9}	8.04	1.83×10^{1}	7.65×10^{-1}	4.52×10^{-10}	6.09×10^{-6}
f7	Mean	$\mathbf{1.53 \times 10^{2}}$	3.93×10^{3}	3.18×10^{3}	1.64×10^{3}	3.22×10^{2}	6.55×10^{2}	8.06×10^{2}	3.43×10^{3}	3.73×10^{3}	1.13×10^{3}
	Std	1.64×10^{-12}	5.63	2.26×10^{1}	1.68	1.07×10^{2}	1.31×10^{2}	2.98×10^{-10}	9.42	1.28×10^{-8}	4.44×10^{-13}
f8	Mean	$\mathbf{6.95 \times 10^{1}}$	1.03×10^{3}	1.05×10^{3}	7.16×10^{2}	2.90×10^{2}	3.87×10^{2}	4.71×10^{2}	1.87×10^{3}	1.57×10^{3}	7.08×10^{2}
	Std	8.81×10^{-13}	2.59	1.90	1.13×10^{1}	4.10×10^{1}	6.35×10^{1}	5.53×10^{1}	4.53	3.17×10^{-9}	9.84×10^{-10}
f9	Mean	$\mathbf{1.44 \times 10^{1}}$	2.41×10^{4}	4.35×10^{4}	2.21×10^{4}	4.62×10^{3}	2.74×10^{3}	1.42×10^{4}	8.38×10^{4}	5.88×10^{4}	1.25×10^{4}
	Std	1.77×10^{-10}	2.64×10^{4}	1.87×10^{4}	1.23×10^{2}	2.36×10^{3}	7.86×10^{3}	4.15×10^{3}	2.29×10^{3}	2.08×10^{-6}	9.58×10^{-8}
f10	Mean	$\mathbf{8.51 \times 10^{3}}$	1.50×10^{4}	1.58×10^{4}	1.60×10^{4}	1.13×10^{4}	1.25×10^{4}	1.20×10^{4}	3.25×10^{4}	2.59×10^{4}	1.50×10^{4}
	Std	1.59×10^{-6}	1.78×10^{3}	2.51×10^{3}	8.29×10^{1}	4.59×10^{3}	1.42×10^{3}	1.46×10^{3}	2.18×10^{2}	7.82×10^{-8}	1.17×10^{1}
f11	Mean	$\mathbf{5.18 \times 10^{2}}$	1.40×10^{4}	8.85×10^{4}	1.50×10^{3}	3.44×10^{3}	9.85×10^{2}	4.49×10^{3}	2.32×10^{5}	1.66×10^{5}	1.40×10^{3}
	Std	2.16×10^{2}	2.98×10^{9}	9.38×10^{3}	1.81×10^{-1}	9.98×10^{-1}	2.29×10^{9}	3.38×10^{-10}	4.89×10^{5}	8.76×10^{-7}	6.83×10^{-3}
f12	Mean	$\mathbf{5.38 \times 10^{5}}$	4.56×10^{10}	2.74×10^{10}	1.98×10^{6}	8.21×10^{9}	1.22×10^{9}	1.41×10^{10}	2.22×10^{11}	1.24×10^{11}	1.42×10^{7}
	Std	5.78×10^{6}	2.63×10^{8}	2.55×10^{7}	1.21×10^{2}	1.82×10^{8}	1.19×10^{10}	5.73×10^{-3}	2.92×10^{6}	1.27×10^{-1}	2.54×10^{4}
f13	Mean	$\mathbf{1.67 \times 10^{3}}$	4.80×10^{9}	4.70×10^{9}	6.25×10^{3}	1.14×10^{9}	1.95×10^{8}	2.55×10^{9}	5.36×10^{10}	2.43×10^{10}	1.43×10^{4}
	Std	4.41×10^{1}	1.62×10^{8}	3.81×10^{6}	1.43×10^{-3}	1.61×10^{8}	9.68×10^{9}	2.59	1.86×10^{6}	3.66×10^{-2}	1.78×10^{-4}
f14	Mean	5.39×10^{4}	1.20×10^{7}	9.72×10^{6}	2.45×10^{5}	4.92×10^{5}	4.01×10^{5}	1.04×10^{6}	1.39×10^{8}	8.13×10^{6}	$\mathbf{4.39 \times 10^{3}}$
	Std	1.98×10^{1}	1.36×10^{7}	1.06×10^{6}	1.61×10^{1}	2.26×10^{7}	8.12×10^{8}	2.73×10^{-7}	1.31×10^{6}	8.34×10^{-5}	7.40×10^{-4}
f15	Mean	$\mathbf{5.34 \times 10^{2}}$	1.50×10^{9}	1.66×10^{9}	2.25×10^{3}	3.32×10^{8}	2.15×10^{7}	5.53×10^{7}	2.95×10^{10}	5.92×10^{9}	6.16×10^{3}
	Std	6.47×10^{1}	4.33×10^{7}	1.15×10^{7}	2.68×10^{-5}	3.71×10^{7}	2.87×10^{9}	3.72×10^{-4}	2.07×10^{6}	1.73×10^{-2}	4.59×10^{-4}
f16	Mean	$\mathbf{1.83 \times 10^{3}}$	6.03×10^{3}	6.18×10^{3}	4.37×10^{3}	2.84×10^{3}	2.74×10^{3}	3.83×10^{3}	2.59×10^{4}	1.17×10^{4}	3.78×10^{3}
	Std	2.26×10^{-6}	2.96×10^{2}	3.76×10^{1}	5.30×10^{1}	1.37×10^{2}	1.99×10^{3}	4.46×10^{2}	1.09×10^{2}	3.33×10^{-8}	1.48
f17	Mean	$\mathbf{9.62 \times 10^{2}}$	2.90×10^{4}	1.11×10^{4}	3.52×10^{3}	3.27×10^{3}	1.99×10^{3}	2.00×10^{3}	1.88×10^{7}	9.93×10^{4}	3.58×10^{3}
	Std	4.45×10^{-6}	3.90×10^{5}	1.38×10^{1}	2.51×10^{1}	2.96×10^{2}	7.09×10^{4}	1.91×10^{2}	6.41×10^{4}	6.47×10^{-7}	2.27
f18	Mean	2.02×10^{5}	1.48×10^{7}	1.22×10^{7}	5.09×10^{5}	1.02×10^{6}	3.65×10^{6}	6.00×10^{6}	3.70×10^{8}	1.61×10^{7}	$\mathbf{2.47 \times 10^{4}}$
	Std	5.01×10^{1}	1.26×10^{7}	2.26×10^{6}	4.74×10^{1}	1.96×10^{7}	2.27×10^{9}	2.87×10^{-4}	1.23×10^{6}	1.25×10^{-4}	4.39×10^{-2}
f19	Mean	$\mathbf{9.17 \times 10^{2}}$	1.14×10^{9}	9.68×10^{8}	1.65×10^{3}	3.93×10^{8}	1.15×10^{7}	9.31×10^{6}	2.93×10^{10}	6.21×10^{9}	4.43×10^{3}
	Std	7.82×10^{-5}	3.45×10^{7}	5.41×10^{6}	2.16×10^{-2}	1.89×10^{8}	1.06×10^{9}	3.00×10^{-2}	2.62×10^{6}	1.71×10^{-2}	1.12×10^{-3}
f20	Mean	$\mathbf{9.85 \times 10^{2}}$	3.13×10^{3}	3.65×10^{3}	3.25×10^{3}	2.06×10^{3}	1.38×10^{3}	1.34×10^{3}	5.84×10^{3}	3.63×10^{3}	2.99×10^{3}
	Std	1.64×10^{-4}	1.55×10^{3}	6.81×10^{2}	3.75×10^{1}	1.63×10^{3}	6.61×10^{2}	6.27×10^{2}	1.80×10^{2}	1.98×10^{-8}	9.64
f21	Mean	$\mathbf{2.92 \times 10^{2}}$	1.44×10^{3}	1.34×10^{3}	8.60×10^{2}	8.66×10^{2}	5.78×10^{2}	7.81×10^{2}	2.93×10^{3}	2.17×10^{3}	9.61×10^{2}
	Std	3.85×10^{-5}	1.25	2.99	1.46×10^{1}	3.11×10^{1}	6.63×10^{1}	1.70×10^{2}	4.16×10^{1}	3.30×10^{-9}	1.08×10^{-2}
f22	Mean	$\mathbf{4.46 \times 10^{3}}$	1.66×10^{4}	1.75×10^{4}	1.85×10^{4}	1.24×10^{4}	9.63×10^{3}	1.30×10^{4}	3.30×10^{4}	2.69×10^{4}	1.59×10^{4}
	Std	3.85×10^{-5}	2.25×10^{3}	1.86×10^{3}	1.46×10^{1}	3.11×10^{1}	1.58×10^{3}	1.34×10^{3}	2.26×10^{2}	7.59×10^{-8}	1.08×10^{-2}
f23	Mean	$\mathbf{6.49 \times 10^{2}}$	2.44×10^{3}	2.37×10^{3}	1.31×10^{3}	2.24×10^{3}	8.31×10^{2}	1.81×10^{3}	4.22×10^{3}	2.83×10^{3}	1.55×10^{3}
	Std	9.13×10^{-1}	1.02	1.68	1.44×10^{2}	3.36×10^{3}	4.34×10^{2}	8.50×10^{2}	6.59×10^{1}	2.75×10^{-9}	2.16×10^{1}
f24	Mean	$\mathbf{8.98 \times 10^{2}}$	3.79×10^{3}	3.62×10^{3}	1.92×10^{3}	3.31×10^{3}	8.87×10^{2}	1.68×10^{3}	7.90×10^{3}	4.65×10^{3}	2.18×10^{3}
	Std	4.33×10^{-2}	3.57	3.72	8.77×10^{-1}	4.20×10^{-2}	3.72×10^{2}	1.71×10^{1}	6.73×10^{1}	3.27×10^{-9}	2.16×10^{-1}
f25	Mean	8.66×10^{2}	2.14×10^{4}	7.96×10^{3}	$\mathbf{8.33 \times 10^{2}}$	1.07×10^{3}	1.02×10^{3}	5.28×10^{3}	2.91×10^{4}	2.38×10^{4}	8.27×10^{2}
	Std	5.37×10^{-2}	1.02×10^{2}	5.67×10^{1}	9.43	4.15×10^{-1}	5.20×10^{3}	9.92×10^{-11}	3.53×10^{-1}	2.58×10^{-8}	6.30×10^{-1}
f26	Mean	$\mathbf{3.46 \times 10^{3}}$	2.83×10^{4}	2.62×10^{4}	1.18×10^{4}	1.08×10^{4}	3.79×10^{3}	1.08×10^{4}	5.32×10^{4}	4.21×10^{4}	1.17×10^{4}
	Std	2.31×10^{2}	3.09×10^{1}	1.55×10^{1}	1.32×10^{-3}	1.16×10^{1}	3.43×10^{3}	2.91	4.09×10^{1}	3.48×10^{-8}	5.42×10^{-1}
f27	Mean	7.13×10^{2}	2.24×10^{3}	1.67×10^{3}	9.60×10^{2}	1.49×10^{3}	7.86×10^{2}	3.33×10^{3}	1.23×10^{4}	3.92×10^{3}	$\mathbf{5.00 \times 10^{2}}$
	Std	6.39×10^{-4}	6.72	8.09×10^{-1}	3.65×10^{-3}	1.98×10^{-5}	4.92×10^{3}	9.51×10^{2}	9.82×10^{1}	3.85×10^{-9}	8.90×10^{-1}
f28	Mean	6.66×10^{2}	2.15×10^{4}	1.61×10^{4}	6.23×10^{2}	2.19×10^{3}	$\mathbf{7.81 \times 10^{2}}$	4.74×10^{3}	2.96×10^{4}	2.87×10^{4}	5.72×10^{2}
	Std	5.94	4.78×10^{1}	1.09×10^{1}	1.17×10^{-4}	1.84×10^{-1}	4.41×10^{3}	4.42×10^{-11}	1.75×10^{1}	2.12×10^{-8}	2.55×10^{2}
f29	Mean	$\mathbf{1.55 \times 10^{3}}$	1.28×10^{4}	8.68×10^{3}	4.15×10^{3}	2.98×10^{3}	2.34×10^{3}	5.75×10^{3}	1.15×10^{6}	2.26×10^{4}	3.51×10^{3}
	Std	1.27×10^{2}	7.49×10^{2}	7.44	3.58×10^{-3}	3.16	1.16×10^{7}	1.34×10^{1}	2.14×10^{3}	8.80×10^{-8}	4.80×10^{-1}
f30	Mean	1.22×10^{4}	3.68×10^{9}	4.40×10^{9}	$\mathbf{9.88 \times 10^{3}}$	1.15×10^{9}	1.50×10^{8}	3.46×10^{8}	4.86×10^{10}	1.09×10^{10}	1.26×10^{5}
	Std	8.83×10^{5}	4.65×10^{7}	2.61×10^{4}	6.94×10^{-2}	2.15×10^{3}	5.37×10^{9}	7.35×10^{2}	3.05×10^{6}	2.30×10^{-2}	4.87×10^{5}

Table 7. Wilcoxon signed-rank test results of ten algorithms versus MSTPSO on CEC 2017 (counts over functions).

Algorithms	MSTPSO VS	10-*D*	30-*D*	50-*D*
DQNPSO	+	27	28	29
	−	0	0	0
	≈	2	1	0
APSO_SAC	+	26	29	29
	−	1	0	0
	≈	2	0	0
EPSO	+	26	24	25
	−	2	3	3
	≈	1	2	1
KGPSO	+	25	28	28
	−	2	1	1
	≈	2	0	0
XPSO	+	29	29	29
	−	0	0	0
	≈	0	0	0
MSORL	+	26	27	29
	−	2	1	0
	≈	1	1	0
DRA	+	29	29	29
	−	0	0	0
	≈	0	0	0
RFO	+	21	27	28
	−	7	2	0
	≈	0	0	1
RLACA	+	16	20	22
	−	11	7	5
	≈	2	2	2

4.2. Performance Comparison Across Different Dimensions

In the 10-dimensional tests, Table 3 reports the mean and standard deviation of solution errors after 51 independent runs. For the unimodal functions f_1 and f_3, the proposed algorithm ranks first on f_3 and second on f_1, with only a small gap from the top, reflecting its effectiveness in solving unimodal problems. For the multimodal functions (f_4–f_{10}), MSTPSO achieves the best results on f_5–f_{10}, which not only demonstrates its excellent global search ability but also highlights the effectiveness of its reinforcement learning strategy and perturbation mechanism in escaping local optima. In the hybrid (f_{11}–f_{20}) and composition functions (f_{21}–f_{30}), MSTPSO also achieves first place on f_{11}, f_{14}, f_{16}, f_{17}, f_{20}–f_{23}, f_{28}, and f_{29}.

In the 30-dimensional case, Table 4 presents the mean and standard deviation of solution errors after 51 runs. MSTPSO ranks second on the unimodal functions f_1 and f_3, slightly behind RLACA. For the multimodal functions (f_4–f_{10}), MSTPSO maintains stable rankings, demonstrating its competitiveness in handling multimodal optimization problems. For the hybrid and composition functions (f_{11}–f_{30}), MSTPSO shows significant improvement; however, f_{14} experiences a performance drop, indicating its sensitivity to dimensionality, while other functions achieve notable gains. Specifically, MSTPSO ranks first on f_{12}, f_{13}, f_{15}, f_{26} and f_{30}, underscoring its strong global search ability and remarkable performance in complex hybrid problems.

In the 50-dimensional case, Table 5 shows the mean and standard deviation of solution errors after 51 runs. MSTPSO achieves a significant improvement on unimodal functions, rising to first place on f_1. For the multimodal functions (f_4–f_{10}), MSTPSO obtains the best results on six functions (f_5–f_{10}) and ranks third on f_4, reflecting its robustness in escaping local optima under complex conditions. For the hybrid and composition functions (f_{11}–f_{30}), MSTPSO achieves overall improvement, ranking first on 14 out of 20 functions, which demonstrates its powerful global search capability in high-dimensional settings.

In the 100-dimensional case, Table 6 presents the mean and standard deviation of solution errors, as well as the average rankings over 51 independent runs. MSTPSO achieves the best result on the unimodal function f_1. For multimodal functions, MSTPSO maintains optimal performance on f_5 to f_{10} and ranks second on f_4. In hybrid and composition functions, MSTPSO maintains stable rankings, while other RL-PSO variants show significant performance degradation, fully demonstrating the robustness and stability of the topology structure in high-dimensional complex optimization tasks. Overall, compared with several state-of-the-art PSO algorithms, the results indicate that MSTPSO ranks first across the 10-*D*, 30-*D*, 50-*D*, and 100-*D* CEC 2017 test suites, highlighting its superior performance.

4.3. Convergence Curves and Dynamic Performance Analysis

To further evaluate the convergence performance of MSTPSO, convergence curves are plotted. It should be noted that due to minor differences in initialization strategies, the early-stage convergence behavior of different algorithms may vary. Figure 5 illustrates the convergence processes for functions f_1–f_{17} in 50 dimensions. The proposed method achieves the fastest convergence on f_1, demonstrating significant advantages in solving unimodal functions through multi-agent reinforcement learning-based parameter training. For multimodal functions, MSTPSO consistently shows the fastest convergence on f_5–f_{10}. For hybrid functions, MSTPSO also converges quickly on f_{11}–f_{13}, f_{15}–f_{17}, with strong capability to escape local optima. These results confirm that MSTPSO possesses strong global search ability and robustness in solving complex hybrid optimization problems.

4.4. Boxplot Analysis and Robustness Evaluation

As shown in Figure 6, the boxplot analysis, indicates that MSTPSO demonstrates varying performance across functions. For unimodal function f_1, it shows strong convergence and stability, approaching the optimal solution quickly with almost no outliers. For multimodal functions f_4, f_6, f_7, and f_{10}, MSTPSO effectively avoids local optima, although a few outliers appear on f_{10}, indicating occasional influence by local traps. For hybrid functions f_{12}, f_{16}, f_{19}, and f_{20}, MSTPSO maintains good solution quality and stability, with only minor fluctuations on f_{19} but avoiding many outliers overall. For composition functions f_{25} and f_{30}, especially under high-dimensional complex conditions, MSTPSO shows strong global search capability, successfully escaping local optima with fast convergence and fewer outliers. In summary, MSTPSO demonstrates excellent performance across different optimization problems, particularly excelling on unimodal, multimodal, and composition functions.

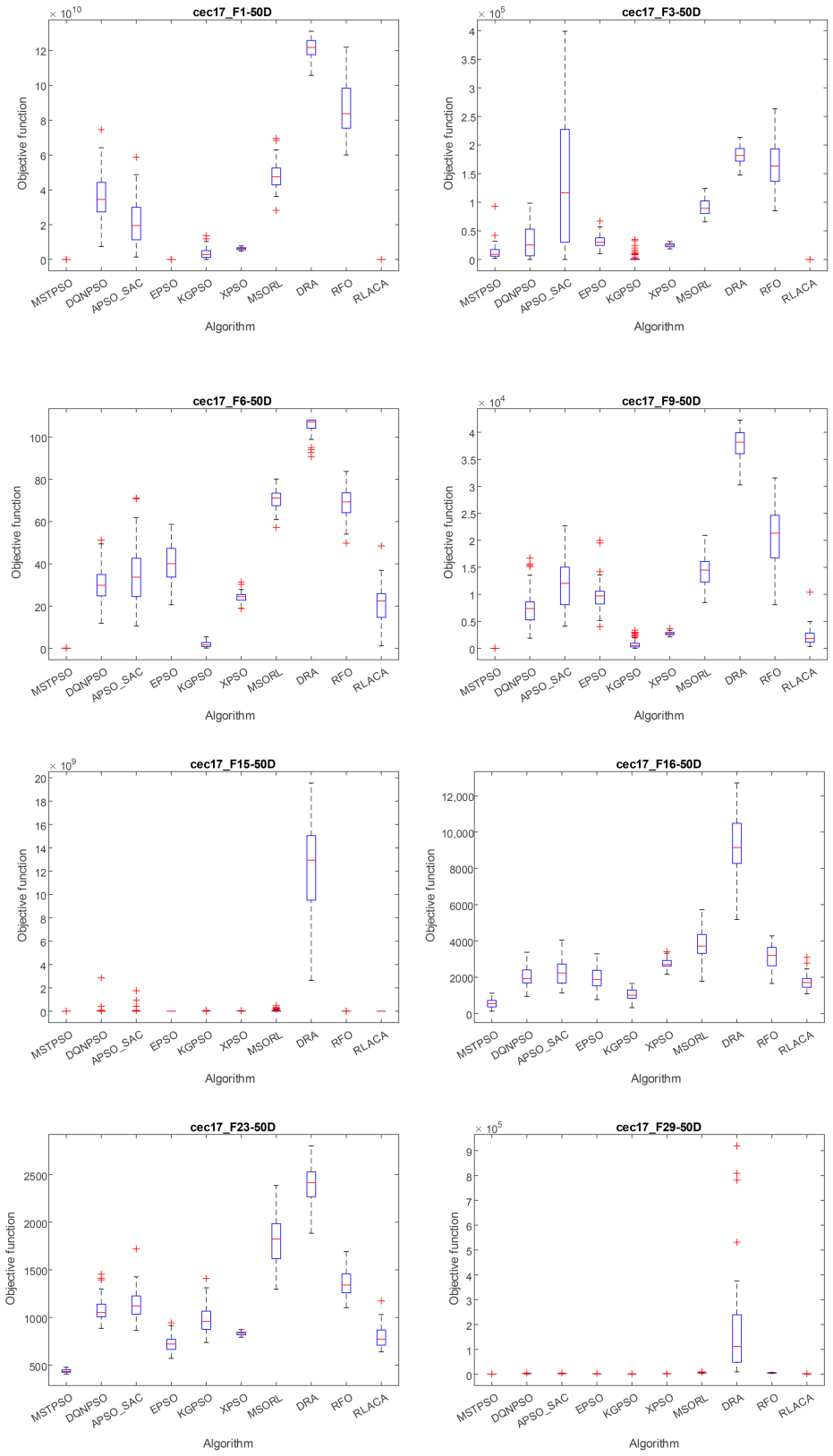


Figure 5. The convergence curves of MSTPSO with PSO variants in the CEC2017 (50-*D*).

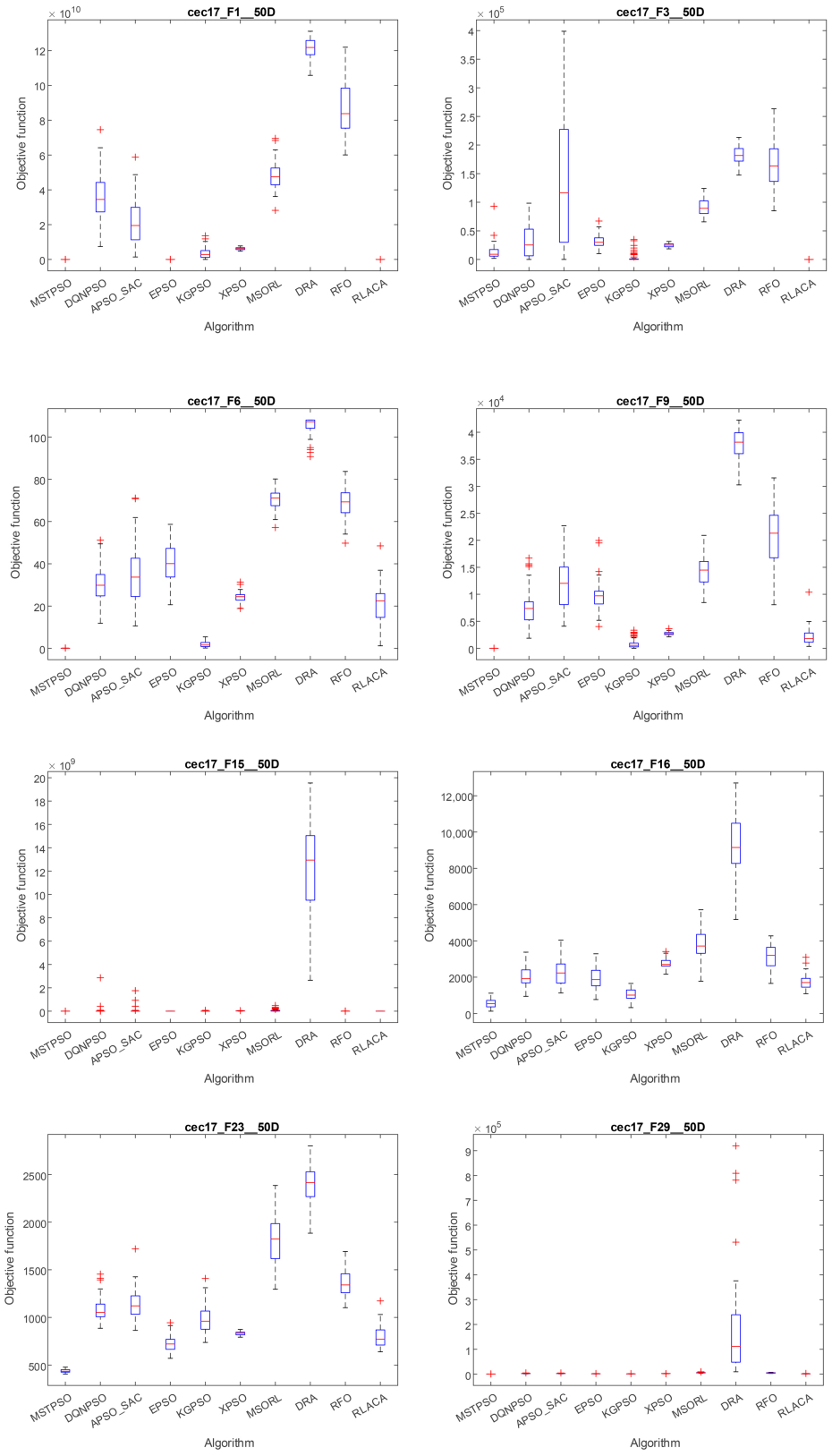


Figure 6. Stability effect of MSTPSO with PSO variants in the CEC2017 (50-*D*).

4.5. Significance Test Analysis (Wilcoxon Rank-Sum Test)

To further verify the statistical superiority of MSTPSO, Table 7 presents the Wilcoxon signed-rank test results of ten algorithms on the CEC2017 benchmark functions under 10-, 30-, and 50-dimensional settings. A significance level of 0.05 was used to determine

whether the performance differences are statistically meaningful, and the *p*-values obtained from the Wilcoxon test indicate the degree of significance. Here, "+" denotes that MSTPSO significantly outperforms the compared algorithm, "-" indicates significant inferiority, and "≈" represents no significant difference. Compared with advanced variants such as DQNPSO, APSO_SAC, XPSO, and DRA, MSTPSO achieved statistically superior results on nearly all test functions, with complete wins across all dimensions against XPSO and DRA. Against EPSO, KGPSO, and MSORL, MSTPSO also maintained clear statistical superiority with only a few non-significant cases. Compared with RFO and RLACA, MSTPSO is slightly better at low dimensions and achieves greater superiority at 30-*D* and 50-*D*. Overall, the Wilcoxon results confirm that MSTPSO significantly outperforms most representative PSO variants across the majority of test functions and dimensions, verifying the statistical reliability of its performance gains.

4.6. Ablation Study on Topology Structures

4.6.1. Multi-Topology Ablation

To verify the contribution of each topology in MSTPSO, three ablation versions are tested: MSTPSO1 removes the FIPS topology, MSTPSO2 removes the small-world topology, and MSTPSO3 removes the exemplar topology. Their optimization performance on the 10- and 30-dimensional CEC2017 functions is compared with the full MSTPSO, as shown in Table 8.

The results show that MSTPSO achieves the best average ranking in both dimensions, confirming the effectiveness of the multi-topology integration in balancing global search and local convergence. The average rankings of MSTPSO are 2.31 in 10-*D* and 2.22 in 30-*D*, significantly better than the ablation versions. MSTPSO1 shows a clear decline in 10-*D*, especially on unimodal and multimodal functions, indicating the critical role of the FIPS topology in maintaining diversity and avoiding premature convergence. MSTPSO2 performs worse in multiple functions, particularly in 30-*D*, with an average rank of 3.81, demonstrating the importance of the small-world topology for global exploration. MSTPSO3 shows performance similar to MSTPSO2 but still inferior to the full version, especially on complex functions. Overall, MSTPSO's superior performance validates the importance of multi-topology integration.

Table 8. Experiments of MSTPSO algorithm and change action algorithm on 10*D* and 30*D*.

Optimization	10-D				30-D			
	MSTPSO	**MSTPSO1**	**MSTPSO2**	**MSTPSO3**	**MSTPSO**	**MSTPSO1**	**MSTPSO2**	**MSTPSO3**
Average	**2.31**	2.82	2.51	2.35	**2.22**	2.18	3.15	2.43
Unimodal	**2.34**	2.90	2.36	2.40	**1.98**	3.81	2.15	2.05
Multimodal	**2.30**	2.42	2.26	2.31	**2.18**	3.24	2.38	2.41
Mixing and compound	**2.35**	2.68	2.60	2.35	**2.12**	3.32	3.29	2.16

4.6.2. Ablation of Stagnation Detection and Hierarchical Response Mechanism

As shown in Table 9, the average ranking results indicate that introducing stagnation detection and the hierarchical response mechanism further improves overall performance in different dimensions. The complete algorithm achieves average ranks of 1.33, 1.43, and 1.47 in 10-*D*, 30-*D*, and 50-*D*, respectively, outperforming the version without this module. The improvement is particularly evident in high-complexity problems. On unimodal functions, both versions converge quickly, but MSTPSO ranks significantly higher in 10-*D* and 50-*D*, showing faster convergence and better escape ability. For multimodal functions, stagnation detection enhances global exploration, and in 30-*D* and 50-*D*, the average rankings are much better than the baseline, indicating higher solution quality and stability in complex spaces. For hybrid and composition functions, the advantage is most significant, with the

complete MSTPSO achieving markedly better average rankings in 10-*D* and 30-*D*. These results confirm that stagnation detection and hierarchical response are key modules for improving MSTPSO's robustness and adaptability in high-dimensional optimization.

Table 9. Friedman ranking of four individual stagnation threshold parameters.

Optimization	30-*D*				50-*D*			
	$T_I = 10$	$T_I = 15$	$T_I = 20$	$T_I = 30$	$T_I = 10$	$T_I = 15$	$T_I = 20$	$T_I = 30$
Average	2.53	2.50	**2.48**	2.49	2.53	2.49	**2.46**	2.52
Unimodal	2.45	2.66	**2.33**	2.56	2.51	**2.42**	2.36	2.71
Multimodal	2.57	**2.33**	2.56	2.54	**2.46**	2.52	2.48	2.54
Mixing and compound	2.52	2.54	**2.47**	**2.47**	2.55	2.49	**2.46**	2.50

To evaluate the impact of stagnation detection thresholds on the stability of the algorithm, a parameter sensitivity study was conducted on the CEC2017 30-*D* and 50-*D* test suites, with the Friedman average rankings shown in Tables 10–12. The individual stagnation threshold T_p, the population stagnation threshold T_i, and the lower velocity bound $V_{\min}$ were set within the ranges 10, 15, 20, 30, 5, 10, 15, 20 and e^{-4}, e^{-5}, e^{-6}, respectively. The experimental results show that the performance of MSTPSO remains stable within these parameter ranges, with only slight fluctuations in the average rankings. The configuration $T_p = 20$, $T_i = 10$ and $V_{\min} = e^{-5}$ achieves the best average rankings in both the 30-*D* and 50-*D* cases.

Table 10. Experiments of MSTPSO algorithm and stop detection module on 10-*D*, 30-*D* and 50-*D*.

Optimization	10-*D*		30-*D*		50-*D*	
	Stagnation	No Stagnation	Stagnation	No Stagnation	Stagnation	No Stagnation
Average	**1.33**	1.67	**1.43**	1.57	**1.47**	1.53
Unimodal	1.73	**1.27**	**1.17**	1.32	**1.38**	1.62
Multimodal	**1.33**	1.67	**1.37**	1.77	**1.52**	1.55
Mixing and compound	**1.30**	1.70	**1.44**	1.55	**1.49**	1.51

Table 11. Friedman ranking of the stagnation threshold parameters for four groups.

Optimization	30-*D*				50-*D*			
	$T_I = 5$	$T_I = 10$	$T_I = 15$	$T_I = 20$	$T_I = 5$	$T_I = 10$	$T_I = 15$	$T_I = 20$
Average	2.53	**2.48**	2.51	**2.48**	2.53	**2.45**	2.49	2.53
Unimodal	2.80	**2.33**	2.35	2.51	2.55	**2.41**	**2.31**	2.72
Multimodal	2.50	**2.51**	2.53	**2.45**	**2.47**	**2.48**	2.44	2.62
Mixing and compound	2.51	**2.49**	2.51	2.49	2.48	**2.43**	2.54	2.55

Table 12. Friedman ranking of lower bound parameters at three speeds.

Optimization	30-*D*			50-*D*		
	$V_{\min} = e^{-4}$	$V_{\min} = e^{-5}$	$V_{\min} = e^{-6}$	$V_{\min} = e^{-4}$	$V_{\min} = e^{-5}$	$V_{\min} = e^{-6}$
Average	2.57	**2.33**	2.56	**2.46**	2.52	2.48
Unimodal	2.52	2.54	**2.47**	2.55	2.49	**2.46**
Multimodal	2.53	2.50	**2.48**	2.53	2.49	**2.46**
Mixing and compound	**2.45**	2.66	2.33	2.51	**2.42**	2.36

5. Conclusions

This paper proposes a multi-topology Particle Swarm Optimization algorithm controlled by Q-learning (MSTPSO). The method introduces a reinforcement learning–driven adaptive topology selection mechanism into the standard PSO framework, enabling particles to dynamically switch among FIPS, small-world, and exemplar-set topologies to

balance global exploration and local exploitation. Meanwhile, a diversity-entropy regulation mechanism is designed to maintain population diversity, and, based on stagnation detection, DE perturbations combined with a global best restart strategy are incorporated to avoid premature convergence and enhance local search capability. Unlike most existing RL-PSO variants, which mainly use reinforcement learning to adapt inertia weight or learning factors, the proposed method focuses on dynamic topology selection and improves performance through the combination of dual-layer experience replay, stagnation detection, and DE perturbations, thus complementing existing approaches in mechanism design. Systematic experiments on the CEC2017 benchmark functions demonstrate that MSTPSO achieves superior performance on unimodal, multimodal, hybrid, and composite functions across four dimensionalities (10, 30, 50, and 100). Compared with various advanced PSO variants, the proposed algorithm attains faster convergence, higher solution accuracy, and better stability, showing particularly strong robustness on high-dimensional complex functions. Further ablation studies reveal that the Q-learning-based topology selection mechanism significantly improves overall convergence quality, while the stagnation-driven DE perturbation and global restart strategy effectively enhance the ability to escape local optima. Nevertheless, some limitations remain. First, the Q-learning parameter settings may require task-specific adjustment and show certain sensitivity. Second, computing diversity entropy and pairwise distance matrices incurs higher computational cost for large populations and high-dimensional problems. Finally, current experiments are limited to benchmark functions, and the applicability of the algorithm to complex constrained optimization and real engineering problems requires further investigation. Future work may proceed in several directions: (1) optimizing the reinforcement learning control strategy to reduce parameter sensitivity and improve efficiency on large-scale, high-dimensional problems; (2) extending the algorithm to more complex scenarios such as constrained, dynamic, and multiobjective optimization to further verify its generality; (3) exploring parallel computing and hardware acceleration methods, such as GPU computing and distributed architectures, to enhance scalability; and (4) applying the algorithm to real engineering tasks—including industrial scheduling, image and vision inspection, and path planning—to validate its practical value.

Author Contributions: X.H., conceptualization; work preparation; S.W., conceptualization; methodology; software; validation; formal analysis; investigation; resources; data curation; visualization; writing—original draft preparation; writing—review and editing; X.L., work preparation; data curation; T.W., conceptualization; supervision; writing—review and editing; project administration; funding acquisition; G.Q., conceptualization; supervision; Z.Z., writing—review and editing. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: Data are contained within the article; our code is released at https://github.com/shenweiwang767-ai/MSTPSO.git, accessed on 15 October 2025.

Acknowledgments: The authors gratefully acknowledge the support from the Scientific Research Project of the Department of Education of Guangdong Province (Grant No. 2022ZDZX3034) and the Key Platform and Research Project for Ordinary Universities of Guangdong Province (Grant No. 2024ZDZX1009). The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Tanabe, R.; Fukunaga, A. Success-history based parameter adaptation for differential evolution. In Proceedings of the 2013 IEEE Congress on Evolutionary Computation, Cancun, Mexico, 20–23 June 2013; pp. 71–78.
2. Liu, Y.; Liu, J.; Ding, J.; Yang, S.; Jin, Y. A surrogate-assisted differential evolution with knowledge transfer for expensive incremental optimization problems. *IEEE Trans. Evol. Comput.* **2023**, *28*, 1039–1053. [CrossRef]
3. Lambora, A.; Gupta, K.; Chopra, K. Genetic algorithm—A literature review. In Proceedings of the 2019 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COMITCon), Faridabad, India, 14–16 February 2019; pp. 380–384.
4. Ngo, L.; Ha, H.; Chan, J.; Nguyen, V.; Zhang, H. High-dimensional Bayesian optimization via covariance matrix adaptation strategy. *arXiv* **2024**, arXiv:2402.03104. [CrossRef]
5. Xiao, S.; Wang, H.; Wang, W.; Huang, Z.; Zhou, X.; Xu, M. Artificial bee colony algorithm based on adaptive neighborhood search and Gaussian perturbation. *Appl. Soft Comput.* **2021**, *100*, 106955. [CrossRef]
6. Shi, Y.; Eberhart, R. A modified particle swarm optimizer. In Proceedings of the 1998 IEEE International Conference on Evolutionary Computation Proceedings, San Diego, CA, USA, 25–27 March 1998; IEEE World Congress on Computational Intelligence (Cat. No. 98TH8360); pp. 69–73.
7. Eberhart, R.; Shi, Y. Particle swarm optimization and its applications to VLSI design and video technology. In Proceedings of the 2005 IEEE International Workshop on VLSI Design and Video Technology, Suzhou, China, 28–30 May 2005; pp. xxiii–xxiii.
8. Ratnaweera, A.; Halgamuge, S.K.; Watson, H.C. Self-organizing hierarchical particle swarm optimizer with time-varying acceleration coefficients. *IEEE Trans. Evol. Comput.* **2004**, *8*, 240–255. [CrossRef]
9. Chen, Q.; Li, C.; Guo, W. Railway passenger volume forecast based on IPSO-BP neural network. In Proceedings of the 2009 International Conference on Information Technology and Computer Science, Kiev, Ukraine, 25–26 July 2009; Volume 2, pp. 255–258.
10. Liu, H.R.; Cui, J.C.; Lu, Z.D.; Liu, D.Y.; Deng, Y.J. A hierarchical simple particle swarm optimization with mean dimensional information. *Appl. Soft Comput.* **2019**, *76*, 712–725. [CrossRef]
11. El-Kenawy, E.S.; Eid, M. Hybrid gray wolf and particle swarm optimization for feature selection. *Int. J. Innov. Comput. Inf. Control* **2020**, *16*, 831–844.
12. Adamu, A.; Abdullahi, M.; Junaidu, S.B.; Hassan, I.H. An hybrid particle swarm optimization with crow search algorithm for feature selection. *Mach. Learn. Appl.* **2021**, *6*, 100108. [CrossRef]
13. Khan, T.A.; Ling, S.H. A novel hybrid gravitational search particle swarm optimization algorithm. *Eng. Appl. Artif. Intell.* **2021**, *102*, 104263. [CrossRef]
14. Lin, A.; Liu, D.; Li, Z.; Hasanien, H.M.; Shi, Y. Heterogeneous differential evolution particle swarm optimization with local search. *Complex Intell. Syst.* **2023**, *9*, 6905–6925. [CrossRef]
15. Lin, A.; Li, S.; Liu, R. Mutual learning differential particle swarm optimization. *Egypt. Inform. J.* **2022**, *23*, 469–481. [CrossRef]
16. Chen, H.; Shen, L.Y.; Wang, C.; Tian, L.; Zhang, S. Multi Actors-Critic based particle swarm optimization algorithm. *Neurocomputing* **2025**, *624*, 129460. [CrossRef]
17. Zhongxing, L.; Fangxi, Z.; Bingchen, L.; Shaohua, X. Quality monitoring of tractor hydraulic oil based on improved PSO-BPNN. *J. Chin. Agric. Mech.* **2024**, *45*, 140.
18. Huang, Y.; Li, W.; Tian, F.; Meng, X. A fitness landscape ruggedness multiobjective differential evolution algorithm with a reinforcement learning strategy. *Appl. Soft Comput.* **2020**, *96*, 106693. [CrossRef]
19. Lu, L.; Zheng, H.; Jie, J.; Zhang, M.; Dai, R. Reinforcement learning-based particle swarm optimization for sewage treatment control. *Complex Intell. Syst.* **2021**, *7*, 2199–2210. [CrossRef]
20. Janson, S.; Middendorf, M. A hierarchical particle swarm optimizer and its adaptive variant. *IEEE Trans. Syst. Man Cybern. Part* **2005**, *35*, 1272–1282. [CrossRef] [PubMed]
21. Zhang, J.; Zhang, J.; Ji, W.; Sun, X.; Zhang, L. Particle swarm optimization algorithm with self-correcting and dimension by dimension learning capabilities. *J. Chin. Comput. Syst.* **2021**, *42*, 919–926.
22. Xia, X.; Gui, L.; He, G.; Wei, B.; Zhang, Y.; Yu, F.; Wu, H.; Zhan, Z.H. An expanded particle swarm optimization based on multi-exemplar and forgetting ability. *Inf. Sci.* **2020**, *508*, 105–120. [CrossRef]
23. Jiang, S.; Ding, J.; Zhang, L. A personalized recommendation algorithm based on weighted information entropy and particle swarm optimization. *Mob. Inf. Syst.* **2021**, *2021*, 3209140. [CrossRef]
24. Engelbrecht, A.P. *Computational Intelligence: An Introduction*; Wiley Online Library: Hoboken, NJ, USA, 2007; Volume 2.
25. Liu, Q.; Van Wyk, B.J.; Sun, Y. Small world network based dynamic topology for particle swarm optimization. In Proceedings of the 2015 11th International Conference on Natural Computation (ICNC), Zhangjiajie, China, 15–17 August 2015; pp. 289–294.
26. Zeng, N.; Wang, Z.; Liu, W.; Zhang, H.; Hone, K.; Liu, X. A dynamic neighborhood-based switching particle swarm optimization algorithm. *IEEE Trans. Cybern.* **2020**, *52*, 9290–9301. [CrossRef]
27. Pan, L.; Zhao, Y.; Li, L. Neighborhood-based particle swarm optimization with discrete crossover for nonlinear equation systems. *Swarm Evol. Comput.* **2022**, *69*, 101019. [CrossRef]

28. Li, T.; Shi, J.; Deng, W.; Hu, Z. Pyramid particle swarm optimization with novel strategies of competition and cooperation. *Appl. Soft Comput.* **2022**, *121*, 108731. [CrossRef]
29. Li, W.; Liang, P.; Sun, B.; Sun, Y.; Huang, Y. Reinforcement learning-based particle swarm optimization with neighborhood differential mutation strategy. *Swarm Evol. Comput.* **2023**, *78*, 101274. [CrossRef]
30. Jiang, C.; Wang, J. Time domain waveform synthesis method of shock response spectrum based on PSO-LSN algorithm. *J. Vib. Shock* **2024**, *43*, 102–107.
31. Tatsis, V.A.; Parsopoulos, K.E. Dynamic parameter adaptation in metaheuristics using gradient approximation and line search. *Appl. Soft Comput.* **2019**, *74*, 368–384. [CrossRef]
32. Liu, Y.; Lu, H.; Cheng, S.; Shi, Y. An adaptive online parameter control algorithm for particle swarm optimization based on reinforcement learning. In Proceedings of the 2019 IEEE Congress on Evolutionary Computation (CEC), Wellington, New Zealand, 10–13 June 2019; pp. 815–822.
33. Yin, S.; Jin, M.; Lu, H.; Gong, G.; Mao, W.; Chen, G.; Li, W. Reinforcement-learning-based parameter adaptation method for particle swarm optimization. *Complex Intell. Syst.* **2023**, *9*, 5585–5609. [CrossRef]
34. Hamad, Q.S.; Samma, H.; Suandi, S.A.; Mohamad-Saleh, J. Q-learning embedded sine cosine algorithm (QLESCA). *Expert Syst. Appl.* **2022**, *193*, 116417. [CrossRef]
35. Hamad, Q.S.; Samma, H.; Suandi, S.A. Feature selection of pre-trained shallow CNN using the QLESCA optimizer: COVID-19 detection as a case study. *Appl. Intell.* **2023**, *53*, 18630–18652. [CrossRef]
36. Wang, F.; Wang, X.; Sun, S. A reinforcement learning level-based particle swarm optimization algorithm for large-scale optimization. *Inf. Sci.* **2022**, *602*, 298–312. [CrossRef]
37. Sutton, R.S.; Barto, A.G. *Reinforcement Learning: An Introduction*; MIT Press: Cambridge, UK, 1998; Volume 1.
38. Watkins, C.J.; Dayan, P. Q-learning. *Mach. Learn.* **1992**, *8*, 279–292. [CrossRef]
39. Qiang, W.; Zhongli, Z. Reinforcement learning model, algorithms and its application. In Proceedings of the 2011 International Conference on Mechatronic Science, Electric Engineering and Computer (MEC), Jilin, China, 19–22 August 2011; pp. 1143–1146.
40. Awad, N.H.; Ali, M.Z.; Suganthan, P.N. Ensemble sinusoidal differential covariance matrix adaptation with Euclidean neighborhood for solving CEC2017 benchmark problems. In Proceedings of the 2017 IEEE Congress on Evolutionary Computation (CEC), Donostia/San Sebastian, Spain, 5–8 June 2017; pp. 372–379.
41. Mendes, R.; Kennedy, J.; Neves, J. The fully informed particle swarm: Simpler, maybe better. *IEEE Trans. Evol. Comput.* **2004**, *8*, 204–210. [CrossRef]
42. Wu, X.; Han, J.; Wang, D.; Gao, P.; Cui, Q.; Chen, L.; Liang, Y.; Huang, H.; Lee, H.P.; Miao, C.; et al. Incorporating surprisingly popular algorithm and euclidean distance-based adaptive topology into PSO. *Swarm Evol. Comput.* **2023**, *76*, 101222. [CrossRef]
43. Aoun, O. Deep Q-network-enhanced self-tuning control of particle swarm optimization. *Modelling* **2024**, *5*, 1709–1728. [CrossRef]
44. von Eschwege, D.; Engelbrecht, A. Soft actor-critic approach to self-adaptive particle swarm optimisation. *Mathematics* **2024**, *12*, 3481. [CrossRef]
45. Yuan, Y.L.; Hu, C.M.; Li, L.; Mei, Y.; Wang, X.Y. Regional-modal optimization problems and corresponding normal search particle swarm optimization algorithm. *Swarm Evol. Comput.* **2023**, *78*, 101257. [CrossRef]
46. Zhang, D.; Ma, G.; Deng, Z.; Wang, Q.; Zhang, G.; Zhou, W. A self-adaptive gradient-based particle swarm optimization algorithm with dynamic population topology. *Appl. Soft Comput.* **2022**, *130*, 109660. [CrossRef]
47. Wang, X.; Wang, F.; He, Q.; Guo, Y. A multi-swarm optimizer with a reinforcement learning mechanism for large-scale optimization. *Swarm Evol. Comput.* **2024**, *86*, 101486. [CrossRef]
48. Mozhdehi, A.T.; Khodadadi, N.; Aboutalebi, M.; El-kenawy, E.S.M.; Hussien, A.G.; Zhao, W.; Nadimi-Shahraki, M.H.; Mirjalili, S. Divine Religions Algorithm: A novel social-inspired metaheuristic algorithm for engineering and continuous optimization problems. *Clust. Comput.* **2025**, *28*, 253. [CrossRef]
49. Braik, M.; Al-Hiary, H. Rüppell's fox optimizer: A novel meta-heuristic approach for solving global optimization problems. *Clust. Comput.* **2025**, *28*, 1–77. [CrossRef]
50. Liu, X.; Wang, T.; Zeng, Z.; Tian, Y.; Tong, J. Three stage based reinforcement learning for combining multiple metaheuristic algorithms. *Swarm Evol. Comput.* **2025**, *95*, 101935. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.